TOWARDS ROBUST LONG-LIVED AGENTS: BENCHMARKING EXPERIENCE LIFECYCLES IN EVOLVING ENVIRONMENTS

Shijue Huang[1*] **Hangyu Guo**[1*] **Hengheng Zhang**[3] **Guo An**[3] **Zhitao He**[1]
Qi Tian[2,3†] **Yiren Feng**[1†]
[1]The Hong Kong University of Science and Technology
[2]Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ)
[3]Huawei Inc.

ReScopeBench.github.io github.com/JoeYing1019/ReScopeBench

ABSTRACT

As agents become more capable, they are moving beyond one-off tasks toward sustained assistance across successive user goals. Such assistance requires agents to discover how unfamiliar environments work, reuse what they learn across tasks, and keep that experience current as environments evolve. Existing benchmarks increasingly study experience reuse and adaptation, but provide limited insight into which parts of accumulated experience agents revise, retain, or lose. To address this gap, we introduce ReScopeBench, a benchmark for the experience lifecycle across 18 interactive environments. It distinguishes structural experience about how an environment is organized and accessed from rule experience about reusable principles and constraints that guide decisions across tasks. To study how experience is acquired, reused, and revised, we propose an experience-first construction approach. It builds task streams around shared experience and introduces silent environmental changes. We evaluate task performance and track changes in saved experience throughout each stream. Across 20 models, we find that retained experience improves task performance, but agents struggle to keep it current as environments change. Even when the required experience is correctly recorded, agents do not always complete tasks successfully. These findings identify selective maintenance and effective use as complementary challenges for long-lived agents.

1 INTRODUCTION

Recent advances have strengthened agents' ability to complete complex tasks in interactive environments (OpenAI, 2026; Anthropic, 2026; Seed, 2025b; Gonzalez-Pumariega et al., 2026; Lin et al., 2026). This progress opens the way to sustained assistance across successive user goals. In an initially unfamiliar environment, such assistance requires the agent to discover how resources are organized and which rules guide its actions. When new requests depend on the same resources and rules, agents can reuse earlier discoveries to guide their work. Continued reuse also requires keeping this experience current when the environment changes.

These requirements make learning from interaction central to sustained assistance. Self-improving agents pursue this goal by using execution and feedback to improve later problem solving (Zhang et al., 2026a;b; Iacob et al., 2026). Benchmarks complement this work by assessing sustained task execution (Xu et al., 2026b; Liu et al., 2026) and the use of experience across tasks. Memory benchmarks evaluate retrieval and updating of information from supplied histories (Maharana et al., 2024; Wu et al., 2025; Tan et al., 2025; Hu et al., 2025). Interactive benchmarks extend this scope to experience that agents acquire through their own actions and reuse on later tasks (Zheng et al., 2025; He et al., 2026; Zhang et al., 2026c; Lei et al., 2026). More recent benchmarks, including

[*]Equal contribution.
[†]Corresponding authors.

Table 1: Comparison of benchmarks for agent memory and experience.

	Experience			Environmental Change		Assessment
Benchmark	Acquisition	Reuse	Budgeted Exploration	Silent	Multiple	Revision & Retention
LoCoMo (Maharana et al., 2024)	Provided	✗	✗	✗	✗	✗
LongMemEval (Wu et al., 2025)	Provided	✗	✗	✓	✗	✗
MemBench (Tan et al., 2025)	Provided	✗	✗	✗	✗	✗
MemoryAgentBench (Hu et al., 2025)	Provided	✓	✗	✓	✓	✗
LifelongAgentBench (Zheng et al., 2025)	Interactive	✓	✗	✗	✗	✗
MemoryArena (He et al., 2026)	Interactive	✓	✗	✗	✗	✗
SkillFlow (Zhang et al., 2026c)	Interactive	✓	✗	✗	✗	✗
SkillEvolBench (Lei et al., 2026)	Interactive	✓	✗	✗	✗	✗
AgingBench (Zhu et al., 2026)	Mixed	✓	✗	✗	✓	✗
EvoArena (Xu et al., 2026a)	Mixed	✓	✗	✓	✓	✗
ReScopeBench	Interactive	✓	✓	✓	✓	✓

AgingBench and EvoArena, examine memory maintenance and adaptation as information and environments evolve (Zhu et al., 2026; Xu et al., 2026a).

Despite these promising advances, it remains less clear which parts of accumulated experience agents revise, retain, or lose as environments change. For example, after a file is moved, an agent may find it again and complete the task without updating the old path in memory. Future tasks may therefore still rely on outdated information. Task success alone does not reveal whether the agent has corrected the experience it will reuse. Evaluation should therefore track accumulated experience across tasks and identify which parts become outdated as the environment changes. The last column of Table 1 asks whether a benchmark separately scores what agents correct and what valid content they retain after the same change.

To support this evaluation, we introduce ReScopeBench, a benchmark spanning 18 initially unfamiliar environments. We focus on two complementary types of experience that agents can acquire in these environments. *Structural experience* describes how resources are organized, related, and accessed. *Rule experience* captures reusable principles and constraints that guide decisions across tasks within an environment. To study how the same experience is acquired, reused, and revised, we propose an *experience-first* construction approach. We first identify structural and rule experience that agents can discover through interaction, then build varied tasks in which agents can reuse it. Local changes invalidate selected parts of this experience while leaving others valid. Changes are unannounced, so agents must detect them through interaction. To examine experience maintenance at different stages of interaction and beyond a single change, we vary change positions and include multiple changes in some streams. Without exploration costs, agents could repeatedly rediscover information instead of maintaining useful experience. We therefore place discovery and execution under environment-specific resource budgets, encouraging agents to reuse valid experience and selectively recheck what may have changed. We evaluate task performance with and without retained experience and assess saved content against private references that reflect the current environment.

Across 20 models, we find that retaining experience improves average task performance, but its acquisition, revision, and use remain uneven. Agents generally record structural experience more completely than rule experience. Our detailed analyses show that, after environmental changes, agents preserve much of the information that remains valid while failing to fully correct outdated information. Execution traces also reveal failures to complete tasks even when the required experience is already correctly recorded. These findings highlight two complementary challenges for sustained assistance: keeping experience current and translating it into successful action.

Our contributions can be summarized as follows:

- We introduce ReScopeBench to study the experience lifecycle in evolving environments. It evaluates task performance together with the acquisition, retention, and revision of saved structural and rule experience.
- We construct 352 tasks across 18 interactive environments, organized into streams in which agents can discover and reuse structural and rule experience. The streams include 26 controlled environmental changes that make selected experience outdated while other experience remains valid.
- Our evaluation of 20 models combines task outcomes, saved experience, and action traces. It reveals gaps in experience acquisition, incomplete revision of outdated content, and failures to complete tasks even when the required experience is correctly recorded.

2 RESCOPEBENCH

2.1 BENCHMARK OVERVIEW

ReScopeBench organizes evaluation as a sequence of tasks in an initially unfamiliar interactive environment. Agents receive task goals and tool interfaces, then discover the resource relationships and operating rules needed to act. They can save this experience in memory and skills for later tasks. Environment-specific resource budgets limit repeated exploration, encouraging reuse of what agents have already learned.

We use *experience-first construction* to study how this experience develops and changes. For each environment, we first identify structural and rule experience that agents can discover through tools and reuse across tasks. We then build tasks with different goals that depend on this shared experience. Local environmental changes invalidate selected parts while leaving others valid. We vary both the positions and the number of changes across task streams. These changes are not announced, so agents must detect them through interaction and revise their experience accordingly. Figure 1 summarizes the construction process and the resulting lifecycle. We evaluate task performance with and without retained experience and assess saved content against private references that reflect the current environment.

2.2 BENCHMARK CONSTRUCTION

As depicted in Figure 1a, construction proceeds through environment preparation, experience discovery, joint stream synthesis, and verification.

Environment Preparation. We implement each environment as a simulated application in code, using application scenarios and public documentation to define its entities, workflows, and operations. Each simulation provides reproducible initial states and tools whose outputs reflect the current state. Tool descriptions explain how to use the interfaces, while local resource relationships and operating conditions must be discovered through interaction. Information gathering and execution incur environment-specific resource costs under a shared allowance or operation-specific limits. These limits make repeated discovery costly and encourage agents to reuse useful experience.

Experience Discovery. During benchmark construction, we inspect environment code and data to identify structural and rule experience that can support multiple task goals. We then interact through the environment tools available to evaluated agents to check how this experience can be discovered from ordinary observations. In the robotic logistics environment, for example, structural experience describes where goods are stored and how to access them, while rule experience captures the handling and acceptance requirements for successful transfers. For each selected item, we record its scope, the observations that reveal it, and the decisions and task outcomes that depend on it. These links form a private dependency graph used to build tasks that reuse the experience and environmental changes that invalidate selected parts of it.

To evaluate what agents save, we organize the selected experience into reference experience units. Each unit contains related statements, called *claims*, about resource relationships or operating rules. For each claim, we define a scoring rubric that specifies what the saved content must establish for the claim to count as correct. These references reflect the environment version at each task and are withheld from evaluated agents. Appendix D.1 illustrates a rubric and its application.

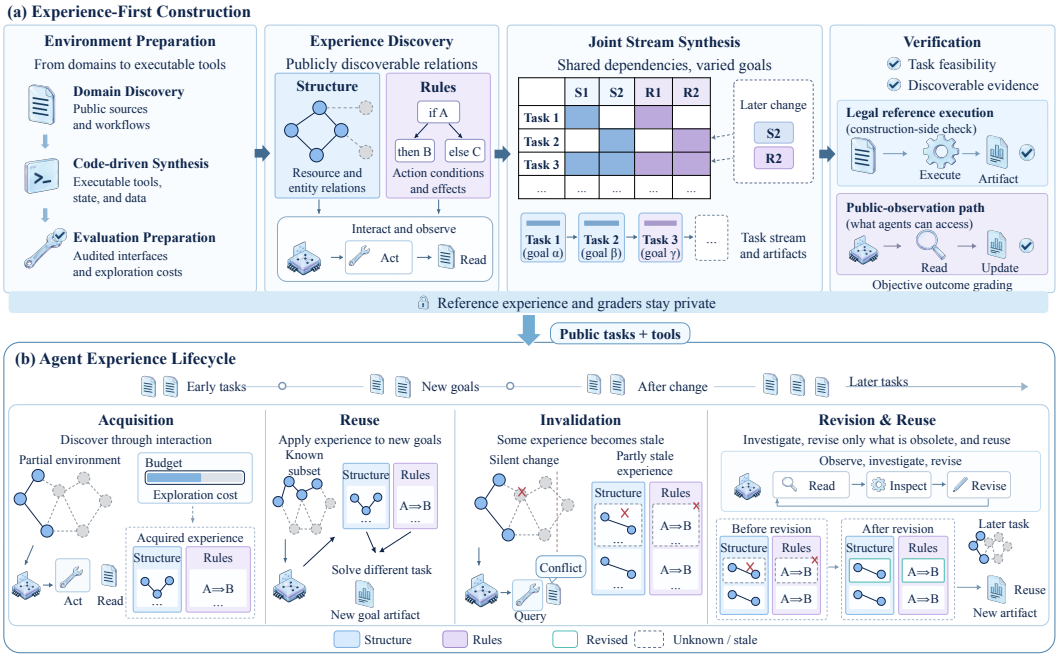


Figure 1: ReScopeBench overview. (a) Experience-first construction connects executable environments, discoverable dependencies, and verified task streams. (b) Agents acquire, reuse, and selectively revise experience after silent changes.

Joint Stream Synthesis. Using the dependency graph, we choose different goals that depend on shared resource relationships and operating rules. The requested objects and conditions vary, so a resource location or operating rule discovered in one task can help with later tasks. At selected task boundaries, we alter some of these relationships or rules and update their reference claims. Later tasks depend on both changed and unchanged relationships or rules, allowing us to assess revision and retention. Changes are not announced in task instructions or boundary messages, but can be detected through tool observations. Their timing and number vary across environments.

We then complete these goals in sequence through the tools available to agents, carrying state forward and applying scheduled changes. The resulting tool-call sequences serve as private reference solutions. Their outputs and final states define each task's delivery requirements. For each task, we implement a private grader, an automated program that scores delivered results against these requirements and accepts equivalent valid outcomes. Appendix D.1 gives an example of task grading. Finally, we compare the resource costs of reference solutions, local correction after changes, and exhaustive searches over candidate resources and rules. Budgets cover the verified completion and correction costs while restricting exhaustive rediscovery.

Verification. Starting from each environment's initial state, we replay the reference solutions in task order and apply the scheduled changes. We check that each solution produces the required outputs within budget and that the task grader assigns the expected score. We also test tool-call sequences that find updated information after a change and support local correction within budget. Finally, we inspect task instructions and tool interfaces for leaked answers and unintended shortcuts. We then freeze the tasks, tools, budgets, change schedules, and graders before evaluating models' ability to acquire, reuse, and revise experience.

2.3 Benchmark Statistics

ReScopeBench contains 352 tasks across 18 environments and nine application domains. Task streams have 16, 24, or 32 tasks and contain one to three environmental changes. The 26 changes occur at different positions, with the first change between tasks 5 and 13. Figure 2 groups these events by change type and environment domain.

We distinguish six mechanisms by what changes in the environment. *Rule* changes alter the conditions governing valid actions and outcomes. *Structure* changes alter resource locations, identifiers, or relationships. *Permission* changes alter which actors are authorized to perform an operation. *Resource* changes alter the availability of items, capacity, or time slots. *Tool-path* changes alter which existing tools or tool sequences can complete an operation. *Mixed* changes combine two or more mechanisms within the same event. These categories describe environmental changes, whereas structural and rule experience describe what agents learn. Appendix A provides the environment inventory and change schedules.

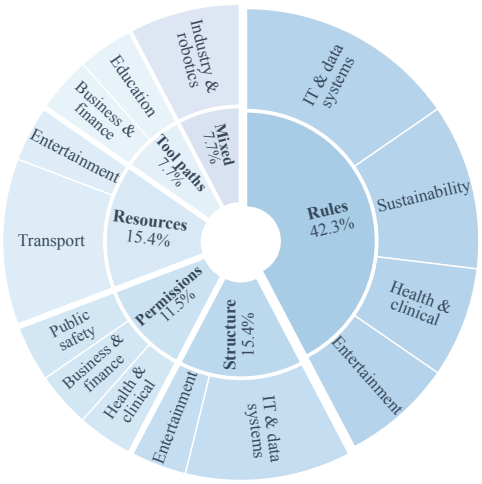


Figure 2: Distribution of 26 change events by change type (inner ring) and environment domain (outer ring).

2.4 Evaluation Protocol and Metrics

2.4.1 Evaluation Protocol

For each model–environment pair, two streams start from the same initial state with identical tasks, tools, budgets, and change schedules. Agents are instructed to store structural experience in memory and rule experience in skills. The *with-experience* condition retains these assets and retrievable history across tasks. The *without-experience* condition clears them between tasks. Both conditions permit within-task learning and start each task with a fresh conversation. Business state persists within each stream and can diverge as agents take different actions.

2.4.2 Metrics

We report three performance metrics for task outcomes and four experience metrics for the coverage, updating, and recovery of saved experience. All scores range from 0 to 100 (higher is better). Appendix B.4 details aggregation.

Task performance. For a fixed model, let $s^+_{e,t}$ and $s^-_{e,t}$ denote task scores with and without experience in environment e at task t. Let T_e be the stream length and $\mathcal{D}_e$ the set of first task positions affected by each change. Baseline performance (BP), experience-enabled performance (EP), and drift-onset performance (DP) are

$$\mathrm{BP}_e = \frac{1}{T_e}\sum_{t=1}^{T_e} s^-_{e,t}, \quad \mathrm{EP}_e = \frac{1}{T_e}\sum_{t=1}^{T_e} s^+_{e,t}, \quad \mathrm{DP}_e = \frac{1}{|\mathcal{D}_e|}\sum_{d\in\mathcal{D}_e} s^+_{e,d}. \tag{1}$$

DP measures performance after executing the first affected task, including adaptation during that task.

Experience coverage. The four experience metrics use post-task memory and skill snapshots from the with-experience stream. Structural coverage (SC) and rule coverage (RC) measure coverage of the corresponding current-version reference inventories. An LLM judge $\mathcal{J}$ receives the entire saved memory and skill corpus together with one reference experience unit. It assesses the unit's claims against their rubrics, regardless of where the content is stored. Each claim scores 1 only if all required and applicable checks pass, and 0 otherwise. SC and RC average claim scores within units, units within each type, and all task positions, then multiply by 100.

Experience updating. For a fixed model and change, consider each affected unit u. Let N_u contain its current-version reference claims for information introduced or changed by this event, and V_u the current-version counterparts of its still-valid claims credited at the last pre-change checkpoint. The nonempty set N_u is shared across models. Both sets remain fixed over an L-task window beginning with the first affected task and ending before the next change or at stream completion. Let

$z_{c,k}$ be binary claim correctness after task $k \in \{1, \ldots, L\}$ in this window. Update coverage (UC) and recovery timeliness (RT) are

$$\mathrm{UC}_u = \frac{100}{|N_u|} \sum_{c \in N_u} z_{c,1}, \qquad \mathrm{RT}_u = \begin{cases} 100(L - \tau_u + 1)/L, & \tau_u \text{ exists}, \\ 0, & \text{otherwise}, \end{cases} \tag{2}$$

where $\tau_u = \min\{k : \forall c \in N_u \cup V_u,\ z_{c,k} = 1\}$. UC measures coverage of the update targets after the first affected task. RT measures how early all update targets and previously correct, still-valid claims are first recorded correctly together. Because update targets do not depend on prior mastery, both metrics can include first acquisition as well as revision.

3 Experiments

3.1 Experimental Setup

We evaluate 20 models from ten families. We compare three compact open-weight models (Qwen3.5-9B, Qwen3.5-35B-A3B, and Qwen3.8-27B) (Team, 2025b) with 17 models offered through commercial services, including models with public weights. The commercial group covers GPT (OpenAI, 2026), Claude (Anthropic, 2026), DeepSeek (DeepSeek-AI, 2026), Doubao (Seed, 2025a), Gemini (Team, 2025a), GLM (GLM, 2026), Kimi (Team, 2026), MiMo (Xiaomi, 2026), MiniMax (Lai et al., 2026), and Qwen3.8-Max. These reports describe the model families. Table 2 lists the versions we evaluated.

All models use the Hermes agent harness (Nous Research, 2026) on the same 18 environments. Each model completes one with-experience stream and one without-experience stream per environment,

Table 2: Main results of ReScopeBench. We evaluate task outcomes and how well agents record and update experience across tasks as their environments change.

Provider	Model	Performance			Experience				Avg.
		BP	EP	DP	SC	RC	UC	RT	
Compact Open-Weight Models									
Alibaba	Qwen3.5-9B	0.51	0.89	0.74	15.14	18.43	1.67	2.95	5.76
Alibaba	Qwen3.5-35B-A3B	0.82	2.75	0.89	18.19	18.21	0.40	4.18	6.49
Alibaba	Qwen3.8-27B	20.07	62.48	38.21	60.90	55.18	29.05	59.17	46.44
Commercial Models									
OpenAI	GPT-5.4	0.09	14.96	12.38	47.75	33.15	12.91	22.85	20.58
MiniMax	MiniMax-M3	5.61	34.50	26.18	45.59	36.63	11.70	35.57	27.97
OpenAI	GPT-6 Luna	20.83	54.69	30.06	56.54	51.09	17.50	49.82	40.08
OpenAI	GPT-5.5	20.55	66.23	30.29	60.34	53.84	17.50	44.88	41.95
ByteDance	Doubao-Seed-2.1-Pro	18.20	64.58	39.88	62.87	59.79	29.75	64.00	48.44
Moonshot	Kimi-K3	22.23	74.08	39.35	65.50	59.72	28.75	64.50	50.59
DeepSeek	DeepSeek-V4-Pro-0813	19.67	70.59	44.75	64.21	63.00	32.44	63.41	51.15
Z.ai	GLM-5.3-Flash	22.35	73.39	42.42	65.76	57.83	32.07	65.62	51.35
OpenAI	GPT-6.1 Sol	24.84	73.97	39.10	71.63	60.66	24.73	64.62	51.36
Google	Gemini-3.8-Flash	25.70	72.47	40.51	65.49	60.56	33.53	65.14	51.91
Xiaomi	MiMo-V2.6-Pro	24.60	73.94	44.73	65.86	61.79	27.93	67.25	52.30
DeepSeek	DeepSeek-V4.1-Flash	19.98	74.73	47.50	68.63	65.18	33.41	65.56	53.57
Alibaba	Qwen3.8-Max	24.09	73.44	46.98	67.56	66.25	31.10	68.64	54.01
Z.ai	GLM-5.3	23.78	75.02	48.81	67.38	66.32	33.00	69.88	54.89
OpenAI	GPT-6 Astra	**25.86**	77.50	42.11	75.68	66.17	29.37	74.12	55.83
Anthropic	Claude Opus 5.5	21.94	74.98	46.75	71.70	**73.22**	33.27	72.89	56.39
Google	Gemini-4-Argon	**25.86**	**81.12**	**50.37**	**76.83**	70.81	**37.63**	**78.72**	**60.19**

following the protocol in Section 2.4. Task outcomes are scored by deterministic graders. We use GPT-6 Astra (OpenAI, 2026) to assess saved experience against the reference rubrics. Appendix B provides inference settings, runtime limits, and detailed execution and scoring procedures.

3.2 Main Results

Table 2 reports the main results. We highlight three observations.

(1) One compact model approaches the commercial median in immediate updating. Qwen3.8-27B records 29.05% of the required updates after the first affected task, close to the commercial median of 29.75%. Its task performance with experience shows a wider gap (62.48 versus 73.44). Both Qwen3.5 models score below 3 on each of these metrics.

(2) Rule coverage remains uneven even among strong models. Structural coverage exceeds rule coverage for 17 of 20 models, with a mean gap of 4.79 points. This imbalance is not confined to models with low coverage. GPT-6 Astra and GPT-6.1 Sol have structural scores above 70, yet their rule scores trail by 9.51 and 10.97 points, respectively. Claude Opus 5.5 reverses this pattern: its rule coverage of 73.22 leads the table and exceeds its structural coverage of 71.70. Strong structural coverage therefore does not uniformly translate into equally complete recording of rules, and the balance between the two types remains model dependent.

(3) High recovery scores can coexist with limited immediate updating. No model exceeds 37.63 in UC, leaving substantial gaps in recorded updates after the first affected task. Model rankings also differ between immediate updating and recovery timeliness. GPT-6 Astra ranks second in RT but tenth in UC, whereas Gemini-3.8-Flash ranks second in UC but ninth in RT. Strong recovery over subsequent tasks should therefore not be taken as evidence of a prompt initial response to change.

3.3 Analysis

We examine experience accumulation, adaptation to environmental change, and the actions that connect saved experience to task completion.

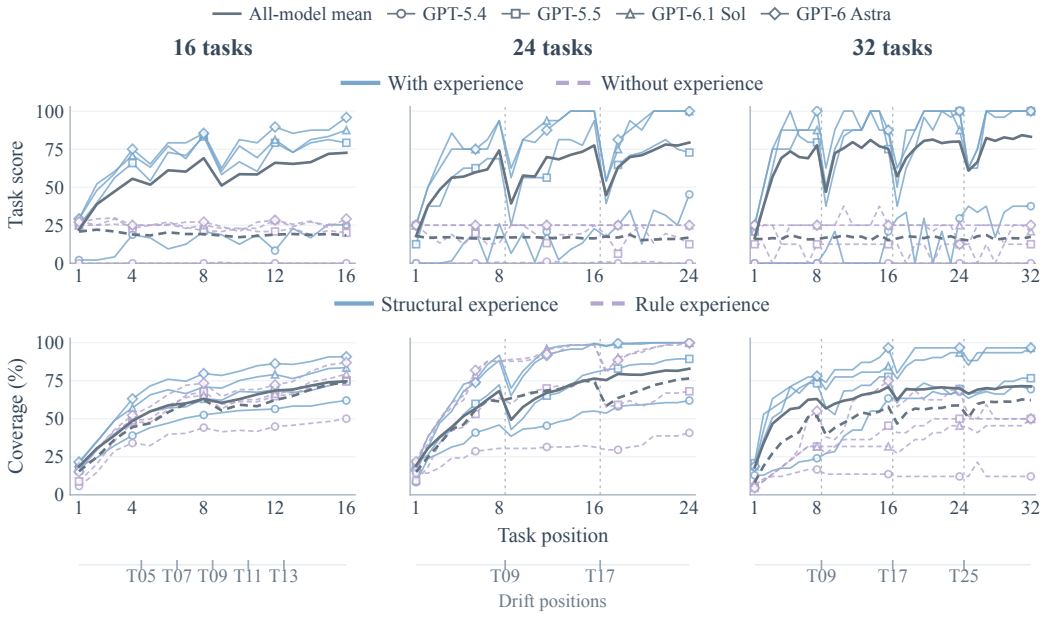


Figure 3: Task performance and experience coverage across task streams.

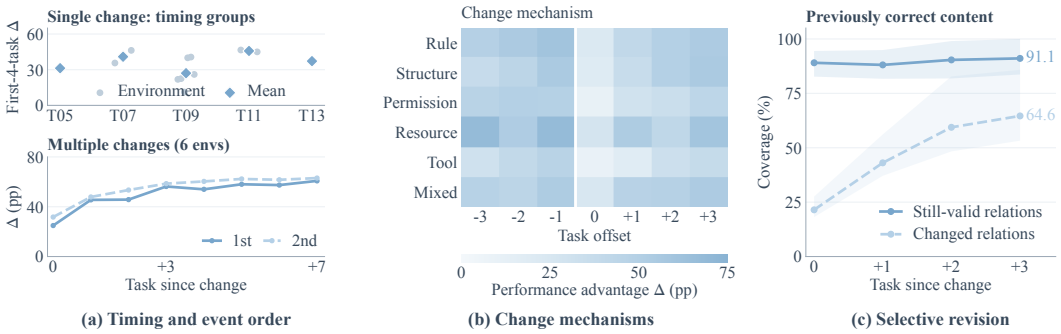


Figure 4: Task performance and selective experience revision under different change contexts.

3.3.1 How Do Experience and Task Performance Evolve Across Tasks?

To examine how experience accumulation relates to task performance, we track task scores and saved content throughout each stream. Figure 3 groups the 18 environments by stream length, using the same environments with equal weight at every position. Dark-gray curves average all 20 models, and markers identify four representative models. Bottom ticks mark change positions across environments, while vertical lines mark onsets shared by the entire cohort.

Performance gains develop across tasks. The upper row shows that performance with experience rises substantially from its initial level across all three cohorts, while the baseline remains low. In the 24- and 32-task cohorts, shared change onsets interrupt this progress, followed by recovery. The 16-task cohort instead combines changes at different positions. The growing separation from the baseline shows that retention benefits develop throughout a stream.

Different experience gaps accompany performance drops. The lower row assesses saved content against the current reference after each task. In the 24-task cohort, task performance falls at both T09 and T17. At T09, mean structural coverage falls while rule coverage stays comparatively stable. At T17, the pattern reverses. Task scores alone therefore do not reveal which type of saved experience no longer matches the environment. Appendix C.1.1 provides the individual environment curves.

Coverage and performance can diverge. GPT-5.4 gains structural coverage across all three cohorts, while its task scores remain much lower overall than those of the other displayed models. Task scores concern the requested outputs, whereas coverage spans the broader reference inventory. Appendix C.1.3 further shows that the association between prior coverage and subsequent performance weakens after accounting for baseline performance, model and task differences, and stream progression. These results motivate examining experience needed by specific requests and how agents act on it, as we do below.

3.3.2 How Do Models Adapt Across Different Change Contexts?

To examine how environmental changes affect the value of experience, we align performance across 20 models to the 26 scheduled events and compare change timing, event order, and mechanism. In Figure 4, Δ is performance with experience minus performance without it, and offset zero denotes the first affected task. We also track previously learned content to distinguish revision from retention.

Larger gains need not mean better recovery. Figure 4a shows no consistent increase in the first-four-task advantage across later-onset groups. Within the same six environments with multiple changes, the second event's eight-task mean advantage is 4.53 points higher than the first event's. The baseline analysis in Appendix C.2.1 subtracts each event's pre-change advantage. The resulting difference ranges from -3.06 to $+2.38$ points across baseline windows. Thus, the higher raw curve does not establish better recovery relative to the advantage already present before change.

Experience benefits survive environmental change. Figure 4b shows a common pattern across all six mechanisms: the mean advantage falls at onset, remains positive, and is higher at the fourth affected task than at onset. The advantage shrinks without reversing sign: with-experience performance remains above the baseline in each mechanism-group mean.

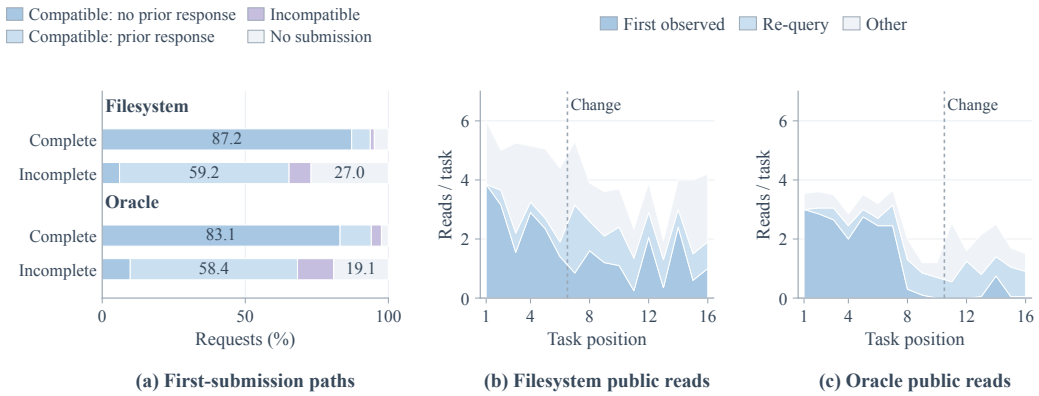


Figure 5: Experience use through first submissions and information gathering in Filesystem and Oracle.

Revision lags behind retention. Figure 4c follows 487 cases across eight environments, each pairing a model with an affected experience unit at a change event. Before the change, each case correctly records every relation selected for updating and at least one relation that remains valid. Over the first four affected tasks, updated-content coverage rises from 21.47% to 64.63%, while still-valid coverage stays between 88.07% and 91.07%. Agents thus retain much of the still-valid content while leaving the corresponding updates incompletely recorded. Bands span the interquartile range of model means. These trajectories reflect both revision and when tasks expose replacement evidence. Appendices C.2.2 and C.2.3 provide the evidence audit and examine whether completed updates persist.

3.3.3 How Are Actions Allocated to Reuse, Exploration, and Rechecking?

To examine how saved experience relates to execution, we match each request's required structural and rule claims to prior content, submitted tool arguments, and delivered artifacts in Filesystem and Oracle. Figure 5a compares 1,304 requests within matched tasks across 18 models. Prior content is complete when all required claims are credited at the preceding checkpoint under the same environment version. Panels b and c then examine how information gathering changes across the full task streams.

Complete experience accompanies compatible first submissions. Compatible first submissions made before receiving successful tool feedback about the request account for 85.14% of requests with complete prior content, compared with 7.92% with incomplete content. The difference is already visible before successful feedback about the request could guide a correction. It is consistent with reuse of prior knowledge, although the recorded sequence does not identify which information source the agent used.

Unsubmitted requests dominate the delivery gap. Figure 5a also shows that requests with incomplete prior content more often receive no submission. To connect these paths to final outcomes, Appendix C.1.2 checks delivered artifacts. Delivery reaches 95.31% with complete prior content and 71.28% with incomplete content. Missing submissions account for 19.42 of this 24.03-point gap. The main difference therefore concerns whether requests reach submission, rather than argument correctness alone.

First observations decline while re-queries increase. For Figure 5b,c, we classify 2,160 business reads across 640 with-experience tasks from all 20 models, including tasks with no reads in the means. A first-observed read reveals a single resource or rule binding absent from earlier tracked observations and from correctly saved experience at the preceding checkpoint under the same version. Re-queries revisit bindings already observed or correctly saved. Other reads include candidate lists, mixed queries, failures, and unmatched queries. Comparing the complete epochs before and after change, first-observed reads decrease from 2.53 to 1.14 per task in Filesystem and from 1.86 to 0.15 in Oracle. Re-queries increase from 0.39 to 1.08 and from 0.47 to 0.84, respectively. Thus, even

as first observations decline, agents continue to query familiar information. These observations do not establish whether each check was necessary. Appendix C.3 gives the classification and resource accounting, with a concrete budget-allocation case in Appendix D.2.

4 Related Work

4.1 Experience-Based Agent Adaptation

Experience-based adaptation methods differ in what they retain from interaction and how they update it. Reflexion and ExpeL turn feedback and trajectories into verbal reflections or reusable insights (Shinn et al., 2023; Zhao et al., 2024). Other methods retain procedures: Voyager stores executable skills (Wang et al., 2024), while Agent Workflow Memory abstracts recurring action sequences into reusable workflows (Wang et al., 2025). To keep guidance current, Dynamic Cheatsheet revises test-time memory as new evidence arrives (Suzgun et al., 2026). ReScopeBench complements these methods with an evaluation of how acquired experience is reused and maintained across tasks as local conditions change.

4.2 Benchmarking Agent Memory and Experience

Benchmarks study experience under supplied histories, interactive tasks, and changing conditions. LoCoMo, LongMemEval, and MemBench evaluate reasoning over supplied histories, with the latter two also assessing updates to remembered information (Maharana et al., 2024; Wu et al., 2025; Tan et al., 2025). MemoryAgentBench adds incremental learning and selective forgetting (Hu et al., 2025). Interactive benchmarks, including LifelongAgentBench and MemoryArena, evaluate experience use across tasks (Zheng et al., 2025; He et al., 2026), while skill-oriented benchmarks study acquisition, refinement, and transfer (Zhang et al., 2026c; Lei et al., 2026; Gao et al., 2026). More recent benchmarks examine maintenance as information and environments change. AgingBench (Zhu et al., 2026) studies memory degradation and revision, including agent-managed workspace scenarios. EvoArena (Xu et al., 2026a) evaluates evolving terminal workflows, codebases, and preferences, and measures whether saved memory contains evidence of user preferences. In ReScopeBench, tasks reuse discoverable structural and rule experience, and local changes make selected parts outdated. We assess saved content alongside task outcomes to distinguish correction of affected relationships from retention of those that remain valid.

5 Conclusion

This paper introduced ReScopeBench, a benchmark for studying experience acquisition, reuse, and revision across 18 interactive environments. Its experience-first construction builds varied tasks around shared structural and rule experience, then introduces silent changes that invalidate selected content while leaving other content valid. Evaluations of 20 models show that retaining experience improves task performance, but rule experience is generally recorded less completely than structural experience. Further analyses show that agents preserve much of the still-valid content while leaving required updates incomplete, and can fail to complete requests even when the required experience is correctly recorded. These findings show why task outcomes and saved experience should be assessed together. ReScopeBench offers a testbed for developing agents that selectively revise outdated experience and effectively apply what they retain.

Acknowledgments

This research was financially supported by the Open Research Fund from Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), under Grant No. GML-KF-26-06.

REFERENCES

Anthropic. Claude Opus 5.5 System Card. Technical report, Anthropic, September 2026. URL https://www.anthropic.com/claude-opus-5-5-system-card.

DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence. *CoRR*, abs/2606.19348, 2026. doi: 10.48550/ARXIV.2606.19348. URL https://doi.org/10.48550/arXiv.2606.19348.

Xingze Gao, Chuanrui Hu, Hongda Chen, Pengfei Yao, Zhao Wang, Yi Bai, Zhengwei Wu, Yunyun Han, Xiaofeng Cong, Jie Gui, Yafeng Deng, and Teng Li. Evoagentbench: Benchmarking agent self-evolution via ability transfer. *CoRR*, abs/2607.05202, 2026. doi: 10.48550/ARXIV.2607.05202. URL https://doi.org/10.48550/arXiv.2607.05202.

GLM. GLM-5: from vibe coding to agentic engineering. *CoRR*, abs/2602.15763, 2026. doi: 10.48550/ARXIV.2602.15763. URL https://doi.org/10.48550/arXiv.2602.15763.

Gonzalo Gonzalez-Pumariega, Vincent Tu, Chih-Lun Lee, Jiachen Yang, Ang Li, and Xin Eric Wang. Scaling agents for computer use. *Trans. Mach. Learn. Res.*, 2026, 2026. URL https://openreview.net/forum?id=eve4jBYa8D.

Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian J. McAuley, Yejin Choi, and Alex Pentland. Memoryarena: Benchmarking agent memory in interdependent multi-session agentic tasks. *CoRR*, abs/2602.16313, 2026. doi: 10.48550/ARXIV.2602.16313. URL https://doi.org/10.48550/arXiv.2602.16313.

Yuanzhe Hu, Yu Wang, and Julian J. McAuley. Evaluating memory in LLM agents via incremental multi-turn interactions. *CoRR*, abs/2507.05257, 2025. doi: 10.48550/ARXIV.2507.05257. URL https://doi.org/10.48550/arXiv.2507.05257.

Alex Iacob, Andrej Jovanovic, William F. Shen, Daniel Burkhardt, Meghdad Kurmanji, Nurbek Tastan, Lorenzo Sani, Niccolò Alberto Elia Venanzi, Ambroise Odonnat, Zeyu Cao, Bill Marino, Xinchi Qiu, and Nicholas D. Lane. The red queen gödel machine: Co-evolving agents and their evaluators. *CoRR*, abs/2606.26294, 2026. doi: 10.48550/ARXIV.2606.26294. URL https://doi.org/10.48550/arXiv.2606.26294.

Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, Jinkai Hu, Jiayao Li, Rui Gao, Zekun Li, Songquan Zhu, Jingkai Zhou, and Pengyu Zhao. Minimax sparse attention. *CoRR*, abs/2606.13392, 2026. doi: 10.48550/ARXIV.2606.13392. URL https://doi.org/10.48550/arXiv.2606.13392.

Yingtie Lei, Zhongwei Wan, Jiankun Zhang, Samiul Alam, Zixuan Zhong, Peizhou Huang, Xin Wang, Jingxuan Zhang, Donghao Zhou, Yunta Hsieh, Zhihao Dou, Hui Shen, Yan Xu, Dimitrios Dimitriadis, Tuo Zhang, and Mi Zhang. Skillevolbench: Benchmarking the evolution from episodic experience to procedural skills. *CoRR*, abs/2605.24117, 2026. doi: 10.48550/ARXIV.2605.24117. URL https://doi.org/10.48550/arXiv.2605.24117.

Yunlong Lin, Zixu Lin, Zhaohu Xing, Biqiang Li, Chenxin Li, Haonan Wang, Haitao Wu, Hengyu Liu, Jianghai Chen, Kaituo Feng, Kaixin Li, Shawn Chen, Shijue Huang, Sixiang Chen, Tsung-Yi Ho, Wenxuan Huang, Xiangyan Liu, Xiaomeng Hu, Xuanhua He, Yan Sun, Yunqing Zhao, Zhiqin Yang, Zehan Wang, Zhengyang Tang, Tianyu Pang, and Xiangyu Yue. Jarvishub: An open harness for canvas-native multimodal creative agents. *CoRR*, abs/2607.23588, 2026. doi: 10.48550/ARXIV.2607.23588. URL https://doi.org/10.48550/arXiv.2607.23588.

Yuxuan Liu, Zhaochen Su, Hangyu Guo, Shijue Huang, Zhixiong Zhang, Kawai Chung, Zhongwei Xie, Tianshi Zheng, Wei Fan, Qing Zong, Haochen Shi, Baixuan Xu, Haoran Li, and Yangqiu Song. Skillhorizon: Constructing long-horizon agent tasks through skill composition, 2026. URL https://arxiv.org/abs/2610.14966.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association*

for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024, pp. 13851–13870. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.ACL-LONG.747. URL https://doi.org/10.18653/v1/2024.acl-long.747.

Nous Research. Hermes Agent. GitHub repository, 2026. URL https://github.com/NousResearch/hermes-agent/releases/tag/v2026.9.14. Version 0.21.3.

OpenAI. GPT-6 Astra System Card. Technical report, OpenAI, September 2026. URL https://deploymentsafety.openai.com/gpt-6-astra.

ByteDance Seed. Seed1.5-thinking: Advancing superb reasoning models with reinforcement learning. *CoRR*, abs/2504.13914, 2025a. doi: 10.48550/ARXIV.2504.13914. URL https://doi.org/10.48550/arXiv.2504.13914.

ByteDance Seed. UI-TARS-2 technical report: Advancing GUI agent with multi-turn reinforcement learning. *CoRR*, abs/2509.02544, 2025b. doi: 10.48550/ARXIV.2509.02544. URL https://doi.org/10.48550/arXiv.2509.02544.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

Mirac Suzgun, Mert Yüksekgönül, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2026 - Volume 1: Long Papers, Rabat, Morocco, March 24-29, 2026*, pp. 7080–7106. Association for Computational Linguistics, 2026. doi: 10.18653/V1/2026.EACL-LONG.333. URL https://doi.org/10.18653/v1/2026.eacl-long.333.

Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. Membench: Towards more comprehensive evaluation on the memory of llm-based agents. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, volume ACL 2025 of *Findings of ACL*, pp. 19336–19352. Association for Computational Linguistics, 2025. doi: 10.18653/V1/2025.FINDINGS-ACL.989. URL https://doi.org/10.18653/v1/2025.findings-acl.989.

Gemini Team. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *CoRR*, abs/2507.06261, 2025a. doi: 10.48550/ARXIV.2507.06261. URL https://doi.org/10.48550/arXiv.2507.06261.

Kimi Team. Kimi K3: open frontier intelligence. *CoRR*, abs/2607.24653, 2026. doi: 10.48550/ARXIV.2607.24653. URL https://doi.org/10.48550/arXiv.2607.24653.

Qwen Team. Qwen3 technical report. *CoRR*, abs/2505.09388, 2025b. doi: 10.48550/ARXIV.2505.09388. URL https://doi.org/10.48550/arXiv.2505.09388.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Trans. Mach. Learn. Res.*, 2024, 2024. URL https://openreview.net/forum?id=ehfRiF0R3a.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, volume 267 of *Proceedings of Machine Learning Research*. PMLR / OpenReview.net, 2025. URL https://proceedings.mlr.press/v267/wang25bx.html.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL https://openreview.net/forum?id=pZiyCaVuti.

LLM-Core Xiaomi. Mimo-v2-flash technical report. *CoRR*, abs/2601.02780, 2026. doi: 10.48550/ARXIV.2601.02780. URL https://doi.org/10.48550/arXiv.2601.02780.

Jundong Xu, Qingchuan Li, Jiaying Wu, Yihuai Lan, Shuyue Stella Li, Huichi Zhou, Bowen Jiang, Lei Wang, Jun Wang, Anh Tuan Luu, Caiming Xiong, Hae Won Park, Bryan Hooi, and Zhiyuan Hu. Evoarena: Tracking memory evolution for robust LLM agents in dynamic environments. *CoRR*, abs/2606.13681, 2026a. doi: 10.48550/ARXIV.2606.13681. URL https://doi.org/10.48550/arXiv.2606.13681.

Kewei Xu, Xiaoben Lu, Shuofei Qiao, Zihan Ding, Haoming Xu, Lei Liang, and Ningyu Zhang. Longds-bench: On the failure of long-horizon agentic data analysis. *CoRR*, abs/2605.30434, 2026b. doi: 10.48550/ARXIV.2605.30434. URL https://doi.org/10.48550/arXiv.2605.30434.

Jenny Zhang, Shengran Hu, Cong Lu, Robert T. Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. In Carl Vondrick, Bharath Hariharan, Colin Raffel, Lerrel Pinto, Diyi Yang, and Aleksandra Faust (eds.), *The Fourteenth International Conference on Learning Representations, ICLR 2026, Rio de Janeiro, Brazil, April 23-27, 2026*. proceedings.iclr.cc, 2026a. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/aa5f5e6eb6f613ec412f1d948dfa21a5-Abstract-Conference.html.

Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob N. Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents. *CoRR*, abs/2603.19461, 2026b. doi: 10.48550/ARXIV.2603.19461. URL https://doi.org/10.48550/arXiv.2603.19461.

Ziao Zhang, Kou Shi, Shiting Huang, Avery Nie, Yu Zeng, Yiming Zhao, Zhen Fang, Qishen Su, Haibo Qiu, Wei Yang, Qingnan Ren, Shun Zou, Wenxuan Huang, Lin Chen, Zehui Chen, and Feng Zhao. Skillflow:benchmarking lifelong skill discovery and evolution for autonomous agents. *CoRR*, abs/2604.17308, 2026c. doi: 10.48550/ARXIV.2604.17308. URL https://doi.org/10.48550/arXiv.2604.17308.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: LLM agents are experiential learners. In Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan (eds.), *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024, February 20-27, 2024, Vancouver, Canada*, pp. 19632–19642. AAAI Press, 2024. doi: 10.1609/AAAI.V38I17.29936. URL https://doi.org/10.1609/aaai.v38i17.29936.

Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, Zhong-Zhi Li, Yingying Zhang, Le Song, and Qianli Ma. Lifelongagentbench: Evaluating LLM agents as lifelong learners. *CoRR*, abs/2505.11942, 2025. doi: 10.48550/ARXIV.2505.11942. URL https://doi.org/10.48550/arXiv.2505.11942.

Jianing Zhu, Yeonju Ro, John T. Robertson, Kevin Wang, Junbo Li, Haris Vikalo, Aditya Akella, and Zhangyang Wang. Your agents are aging too: Agent lifespan engineering for deployed systems. *CoRR*, abs/2605.26302, 2026. doi: 10.48550/ARXIV.2605.26302. URL https://doi.org/10.48550/arXiv.2605.26302.

APPENDIX CONTENTS

A **Benchmark Environments and Changes** 15

B **Evaluation Details** 18

Models, paired streams, experience access, execution, and metrics

B.5 Cross-judge consistency 20

C **Supplementary Analyses** 21

C.1 Experience and task performance 21

Environment trajectories, request outcomes, associations, and acquisition

C.2 Environmental change and experience maintenance 26

Change contexts, selective revision, and persistence

C.3 Interaction and action allocation 29

D **Illustrative Case Studies** 30

Local revision and reuse, and use of saved experience under a budget

Table 3: Goals and stream lengths (T) for all 18 environments. Goals specify required outcomes rather than solution paths.

Environment	T	Task goal
API Reliability	16	Restore failed provider requests to durable accepted results without duplicate effects.
Backup and Recovery	16	Create active asset-protection assignments with daily schedules, activated policies, and site-compatible repositories.
Carbon Credits	32	Complete carbon-credit retirements using currently eligible holdings and required audit evidence.
Cinema Booking	24	Complete paid bookings satisfying the requested film, venue, showtime, ticket type, and seat zone, using eligible pricing and available seats.
Cloud Data Warehouse	24	Commit governed query deliveries with materialized tables, completed exports, valid lane authorities, and applicable approved controls.
Course Registration	16	Create active student registrations in the requested sections and processing lanes under the applicable policy.
Emergency Response	16	Dispatch authorized, available resources. Release each assignment and verify that the resource is available again.
Event Ticketing	24	Complete paid orders with the requested admission class, two distinct available seats, the eligible house offer, and issued tickets.
Filesystem	16	Materialize manifest indexes in their exact eligible delivery lanes under current workspace and delivery-policy bindings.
Kubernetes Operations	24	Publish deployments and services with replicas in the required zones, current namespace-assigned worker pools, and compliant rollout/service settings.
Medical Appointments	16	Schedule policy-compliant follow-ups linked to completed encounters, using currently eligible slots.
MES Order Closeout	16	Close production orders after satisfying material-issue, operation, equipment, and quality preconditions.
Oracle Clinical One	16	Correct completed forms while preserving the supplied values and policy-compliant audit records.
Railway Ticketing	32	Issue new confirmed bookings matching the passenger, route, date, seat class, and requested seat position.
REDCap	16	Resolve record work packets by verifying corrected fields, closing queries, completing both forms, and locking the record event.
Robot Logistics	16	Deliver parcels for priority-ordered work orders, close each order, and return the empty robot to the service hub.
SAP S/4HANA	16	Authorize purchase-to-pay routes, receive outstanding quantities, create receipt-backed supplier invoices, and post them in open company periods.
Supply Chain Finance	16	Finance eligible invoice balances under current program authority, then complete settlement and matched reconciliation.

A Benchmark Environments and Changes

This appendix expands the environment inventory in Section 2.3. Table 3 lists task goals and stream lengths for all 18 environments, totaling 352 task positions before replication across models. Table 4 lists all 26 scheduled changes in the evaluated versions. Each change position denotes the first affected task.

A.1 Task Goals and Stream Lengths

A.2 Complete Change Catalog

The catalog uses the six change mechanisms defined in Section 2.3. We classify events by their effect on environment operations, rather than the names of modified fields. For example, Cinema's replacement pricing plans revise rules, whereas Railway's replacement train services change available resources.

Table 4: All 26 scheduled changes. Task denotes the first affected task position.

Environment	Task	Mechanism	Implemented business change
API Reliability	9	Rule	Revise recovery profiles for four operation-specific cases: queued replays, provider errors, timeouts, and rate limits. Retain the retry action.
Backup and Recovery	5	Rule	Replace protection-route and repository-profile requirements for critical/restricted, standard/confidential, archive/internal, and regulated/public service-tier/data-class pairs.
Carbon Credits	9	Rule	Revise project eligibility and claim protocols for Gold Standard customer allocations and national voluntary claims. Retain holdings.
Carbon Credits	17	Rule	Revise an allocation policy and again amend the national voluntary-claim policy. Retain the earlier Gold Standard allocation rule.
Carbon Credits	25	Rule	Revise Gold Standard allocation and national voluntary-claim policies with new protocols. Retain the allocation rule changed at task 17.
Cinema Booking	9	Resource	Cancel showtimes in three screening groups and open replacements. Transfer screening-specific pricing assignments without changing fare terms.
Cinema Booking	17	Rule	Change accepted checkout channels in three pricing-policy cells. Retire old pricing plans and bind screenings to successor plans.
Cloud Data Warehouse	9	Structure	Rebind delivery lanes to new routing/lineage authorities and destination tuples. Preserve delivery policies and tools.
Cloud Data Warehouse	17	Rule	Introduce region-qualified sensitive-data controls: sensitive EU destinations require compressed-Parquet snapshots. Retain domestic profiles.
Course Registration	9	Tool-path	Change selected workflows to queue then promote, provisionally register then finalize grading basis, or review the profile then register directly.
Emergency Response	9	Permission	Replace recognized responsible/requesting agencies in selected authority domains. Previous requesters no longer authorize dispatch.
Event Ticketing	9	Structure	Rebind ticket types to price tiers and migrate event, performance, and ticket-type identifiers. Preserve seats, purchases, and commercial entitlements.
Event Ticketing	17	Rule	Revise ticket-type eligibility for commercial discount/offer profiles. Retain the migrated catalog identifiers.
Filesystem	7	Structure	Replace selected destination-lane nodes with successor handles for the same delivery purposes. In Grove, only the amber lane changes.
Kubernetes Operations	9	Structure	Reassign namespace-zone allocations to other existing worker pools and rekey namespaces/references. Preserve workloads, capacity, and policies.
Kubernetes Operations	17	Rule	Revise rollout strategy, service type, external-DNS, and NodePort requirements in two policy cells. Retain namespace-to-pool assignments and tools.

Continued on the next page

Table 4 (continued)

Environment	Task	Mechanism	Implemented business change
Medical Appointments	11	Rule	Revise care-plan-update routing for standard, chronic-care, senior-care, and maternity patients, changing continuity/team and in-person/virtual requirements.
MES Order Closeout	7	Mixed	Quarantine old material lots and allocate replacements. Separately revise material-issue quantities for four closeout templates.
Oracle Clinical One	11	Rule	Revise correction audit-mode/reason profiles for screening/transcription, scheduled/clarification, completion/unit normalization, and dispensation/coding alignment visit/correction pairs.
Railway Ticketing	9	Resource	Cancel generation-0 services for the first two route groups (A). Activate generation-1 replacements with their own seat inventories.
Railway Ticketing	17	Resource	Replace generation-0 services for the other two route groups (B) with generation-1 services and their own seat inventories. Retain group A services.
Railway Ticketing	25	Resource	Replace group A's generation-1 services with generation-2 services and their own seat inventories. Retain group B services and issued tickets.
REDCap	9	Permission	Deactivate old project-user authorities and activate successors. Transfer query assignments and execution authority to them.
Robot Logistics	9	Mixed	Rotate local intake/transfer policies and relocate selected groups of parcels, updating parcel locations, future source rooms, room settings, and navigation/pick locations.
SAP S/4HANA	9	Tool-path	Switch the authorization tool and route reference for four short-payment-horizon purchasing routes. Retain receipt, invoicing, and posting workflows.
Supply Chain Finance	13	Permission	Replace the financing authority bundle: funding scope, approval chain, delegation, authority namespace, and submission channel. Retain the financing tool.

B Evaluation Details

B.1 Evaluated Models

We evaluate 20 models from ten families. Table 2 lists the versions, and Section 3 describes the model groups.

Fourteen models explicitly request temperature 1 and top-p 0.95. Claude Opus 5.5, GPT-5.4, GPT-5.5, and the three GPT-6 configurations omit these fields. The three GPT-6 Responses configurations also omit the request seed. Neither omitted sampling fields nor plan-level seeds establish deterministic generation. Reasoning controls differ across providers, so their settings do not establish equal inference compute.

B.2 Paired Streams and Experience Access

All 20 models use the same 18 versioned environments: twelve 16-task streams, four 24-task streams, and two 32-task streams. Each model completes 352 tasks and encounters 26 changes per condition, yielding 360 paired evaluations, 7,040 paired task positions, and 14,080 task outcomes. GPT-5.5 also supported environment development. Change schedules are fixed across models, with observation windows defined separately for each change.

Both conditions start from the same business state and empty experience, with fixed model weights and a fresh conversation for each task. The with-experience condition retains naturally produced memory, skills, and retrievable business history. The without-experience condition clears these assets and other non-business cross-task context, but retains its own business state and environment-managed capability records. Both permit within-task learning and use the same tasks, tools, and exploration budgets. Their business states evolve independently through their actions. Each model–environment pair contributes one stream per condition, with no repeated independent trials.

The Hermes memory and skill interfaces are pinned to revision `345cd2b057a45`. Main memory is limited to 16,000 characters and rejects writes that exceed this limit. Skill content is limited to 100,000 characters. At task start, the agent receives a memory snapshot and a skill index, with full skill contents available on demand. Subsequent writes update storage and may be visible through tool responses. The next task loads the latest retained snapshot. No reference experience is preloaded. The shared prompt recommends memory for structural experience, such as concrete bindings, and skills for rule experience, such as conditional rules. Our experience types are defined by content, regardless of storage location.

Four read-only history tools retrieve completed-task records from the agent's own stream. Records include task requests, business-tool arguments, direct observations including failures, and permitted public outcome feedback. They exclude assistant reasoning and final narratives, previous retrieval results, and memory/skill traffic. Retrieval uses deterministic lexical matching and pagination, without semantic reranking or a fixed last-k cutoff. The tools share an allowance of 30 invocations per task across execution and review. Requests accepted for processing count even if empty or invalid. Once the allowance is exhausted, queries return no content until the next task. History and experience operations are accounted for separately from business exploration costs.

B.3 Execution, Review, and Feedback

Each task permits 100 foreground agent iterations and at most 65,536 output tokens per model response. An iteration may contain multiple tool calls, executed in order with separate state transitions and charges. Business costs follow each environment's declared mechanism. At the iteration limit, the attained business state is graded. Agents see no wall-clock countdown. Infrastructure timeouts apply separately.

Agents are instructed to retain useful evidence, preserve uncertainty, and revise contradicted entries. They can update experience during execution. Background review runs only when the conditions below are met. It uses the same learner model and provider, receives the foreground conversation and remaining history allowance, and cannot execute business actions. It has a separate limit of 100 iterations and a cumulative input-token guard of 600,000.

Skill review requires ten tool iterations, skill-management availability, a final response, and an uninterrupted turn. Memory review instead requires ten user turns within one conversation. Because each task starts a new conversation, reaching the tenth task does not trigger memory review. Skill-only review does not automatically grant memory-write access. Agents can still revise experience during foreground execution.

The runner completes review before grading the terminal state and saving experience, so the current task's grade cannot supervise its review. Later history queries may expose permitted scores and completion counts, but never private reference bindings or grading answers.

B.4 Metrics and Aggregation

BP and EP average all task scores, including the initial task. DP averages with-experience scores on the first task affected by each change, including adaptation within that task. It is neither a change from the preceding score nor a score on the next task. For environments with multiple changes, we average these onset scores within each environment before averaging across environments.

SC and RC measure how much of the full current-version core inventory appears correctly in the with-experience stream's saved textual memory and skills. The reference inventory includes claims the agent has not yet encountered. The corpus includes retained textual backups regardless of extension, but excludes system usage files, curator ledgers, and previous conversation databases. Reference units group related claims independently of file boundaries to preserve context. For experience type $q \in \{S, R\}$ after task t in environment e, let $\mathcal{U}^q_{e,t}$ be the reference units and $\mathcal{C}_{e,t,u}$ the claims in unit u. With binary claim decisions $z_{e,t,c}$, checkpoint coverage is

$$C^q_{e,t} = \frac{100}{|\mathcal{U}^q_{e,t}|} \sum_{u \in \mathcal{U}^q_{e,t}} \frac{1}{|\mathcal{C}_{e,t,u}|} \sum_{c \in \mathcal{C}_{e,t,u}} z_{e,t,c}. \tag{3}$$

For Robot's two-claim source unit, a correct station but incorrect storage handle gives 50% coverage. SC and RC average checkpoint coverage across all task positions. All models use the same references and rubrics for a given environment version.

GPT-6 Astra judges one complete reference unit per request against the complete saved corpus. Model-identifying metadata is withheld, and the judge uses medium reasoning effort and a 14,000-token output cap. Rubric items are checks within claims, not separate requests or weights. A claim receives credit only if every required and applicable item passes. The judge returns item-level findings, reasons, and supporting or contradicting text spans. We validate the output structure and cited line ranges, then compute and aggregate binary claim scores programmatically. Equivalent wording and supported deductions are accepted. Unresolved contradictory alternatives receive no credit merely because one matches the reference. The scores measure correct coverage of the reference inventory, not whole-notebook precision or use of the recorded content.

UC and RT assess updates to each unit affected by a change. We map claims across versions by business conditions and relationships, accounting for splits, replacements, and additions. For each affected unit u, N_u contains the current-version update targets defined by the reference and is shared across models regardless of prior mastery. The model-specific set V_u contains the current-version counterparts of this unit's still-valid claims credited at the last pre-change checkpoint. Both sets remain fixed during the L-task observation window, which begins with the first affected task and ends before the next change or at stream completion. For $k \in \{1, \ldots, L\}$, let $z_{c,k}$ denote binary claim correctness after task k in this window. The first joint completion is $\tau_u = \min\{k : \forall c \in N_u \cup V_u,\ z_{c,k} = 1\}$. Then

$$\mathrm{UC}_u = \frac{100}{|N_u|} \sum_{c \in N_u} z_{c,1}, \qquad \mathrm{RT}_u = \begin{cases} 100(L - \tau_u + 1)/L, & \tau_u \text{ exists}, \\ 0, & \text{otherwise}. \end{cases} \tag{4}$$

Only units with nonempty N_u participate. If V_u is empty, completion covers updates only and makes no claim about preservation. RT requires all claims in $N_u \cup V_u$ to be correct together within the unit, rather than averaging when individual claims first become correct. Completion on the first affected task scores 100, completion on the last scores $100/L$, and no completion scores zero. RT measures timing within this observation window. Later regression does not change the first-completion score.

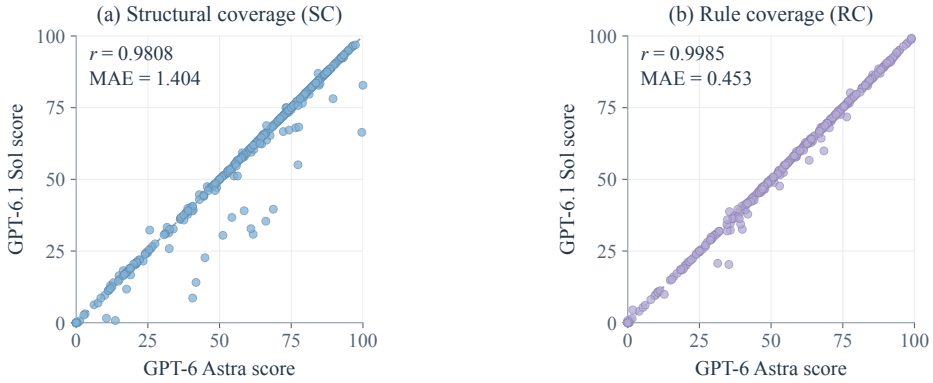


Figure 6: Agreement between GPT-6 Astra and GPT-6.1 Sol experience scores across 360 model–environment pairs per panel. The dashed line denotes equal scores.

UC and RT can include first acquisition as well as revision. RT also depends on pre-change mastery through V_u and should be read alongside SC and RC. Timing starts at the environmental change under the actual task schedule, so it reflects both opportunities to encounter a target and successful updating. It does not isolate revision speed under equal exposure.

All seven metrics weight environments equally. SC and RC average claims within units, units within each type, and task positions within each environment. UC and RT average affected structural and rule units together within each event, then events within each environment. Longer streams, larger inventories, and more frequent changes do not increase an environment's aggregate weight. All models have complete judgments for the same 18 environments. Avg. is the mean of the seven unrounded metrics and serves as a descriptive summary. Correlations across this model set are descriptive associations.

In the supplementary analyses, W_{met} and B_{met} denote task scores with and without experience for model m, environment e, and task t. Their difference $\Delta_{met} = W_{met} - B_{met}$ is distinct from DP. S_{met} and R_{met} denote structural and rule coverage after task t. These curves describe how experience coverage and task performance develop together. Complete-stream statistics average tasks within each model–environment pair, then weight environments and models equally. Task-position curves use a fixed set of environments and all 20 models within each stream-length group. The 16-task group has different change schedules and is not event-aligned. Model interquartile ranges show cross-model dispersion, not repeated-run uncertainty. Missing or inapplicable observations are excluded rather than assigned zero.

B.5 Cross-Judge Consistency

We test sensitivity to the scoring model by rejudging the complete experience corpus from all 20 models and 18 environments. We change the requested judge from GPT-6 Astra to GPT-6.1 Sol through the same provider. The comparison covers 7,040 checkpoints, 121,717 distinct unit-level requests, and 433,850 request-level claim decisions. Saved content, version-specific references, rubrics, prompts, output schema, and inference settings remain fixed. Each request contains the complete saved corpus and one complete reference unit. The second judge does not see the first judge's decisions. Two responses with invalid citations are retried with identical inputs, and all valid responses are retained. Learner trajectories and task-performance scores are unchanged.[1]

Most model–environment scores lie close to the equality line in Figure 6. The judges agree on 223,900 of 226,648 structural claim decisions (98.79%, Cohen's $\kappa = 0.9756$) and 205,637 of 207,202 rule claim decisions (99.24%, $\kappa = 0.9847$). These decisions come from deduplicated requests and are not independent observations. Table 5 reports score agreement before and after

[1] All requests specify GPT-6.1 Sol without model fallback. Of valid responses, 81.0% carry an explicit Sol deployment label and 19.0% an opaque provider identifier. On the explicitly labeled subset, structural/rule claim agreement is 98.76%/99.20%.

Table 5: Cross-judge agreement under identical rubrics. r is Pearson correlation, ρ is Spearman rank correlation, and MAE is mean absolute error in score points.

	360 model–environment pairs		20 model averages		
Metric	r	MAE	r	ρ	MAE
SC	0.9808	1.404	0.9984	0.9805	1.231
RC	0.9985	0.453	0.9998	0.9940	0.337
UC	0.9905	0.469	0.9963	0.9759	0.469
RT	0.9973	0.476	0.9995	0.9865	0.423

equal averaging over environments. Sol scores are lower on average by 1.22 points for SC and 0.30 for RC. The aggregate structural–rule gap remains positive at 4.79 points with Astra and 3.86 with Sol. The judges yield similar model rankings and the same aggregate finding that structural coverage exceeds rule coverage, with small shifts in average scores.

Agreement varies across environments. For SAP and MES, the mean absolute SC differences are 15.18 and 3.75 points, with maximum pairwise differences of 33.20 and 30.73. Across the 20 model averages, SC and RC ranks move by at most three and two positions. We recompute UC and RT from the new claim labels, with each judge determining its own pre-change retained set for RT as required by the metric. This comparison supports aggregate consistency between these GPT-family judges under a shared rubric, but does not independently validate their accuracy.

C Supplementary Analyses

This section provides additional evidence for the three analyses in Section 3.3: experience and task performance, adaptation to environmental change, and action allocation. Complete-stream analyses cover all 20 models and 18 environments. Request-level analyses use Filesystem and Oracle, where verified mappings connect required experience, saved content, and observable actions.

C.1 Experience and Task Performance

C.1.1 Complete Per-Environment Trajectories

Figures 7 and 8 show the environment-level variation behind the aggregate trends. They plot complete task streams, with markers for GPT-5.4, GPT-5.5, GPT-6.1 Sol, and GPT-6 Astra as in Figure 3. Dark-gray curves equally average the same 20 models at every position. The first figure shows task performance in both conditions. The second shows post-task structural and rule coverage in the with-experience condition, measured against the current reference. Vertical lines immediately precede each change's first affected task.

C.1.2 Task-Required Content and Request Outcomes

To relate saved experience to delivery, we align each request's required structural and rule bindings with saved claims, submitted arguments, and delivered artifacts. Figure 5a summarizes the comparison. The trace cohort covers 640 with-experience tasks, 2,560 requests, and 8,826 foreground tool calls with matching responses across all 20 models in Filesystem and Oracle. We select these environments for their verified mappings, rather than as a random sample.

A request at task t has *complete prior experience* when every required structural and rule claim is credited at checkpoint $t-1$ under the same reference version. Credit requires all required and applicable rubric checks to pass. Prior experience is incomplete if any mapped claim lacks credit. The 320 requests on initial and change-onset tasks have no same-version predecessor, so their prior status is unknown. The remaining 2,240 requests comprise 1,014 complete-prior and 1,226 incomplete-prior requests.

We compare complete and incomplete priors within each model's tasks that contain both groups and have resolved submission attribution. This yields 326 model/environment/task cells, 1,304 requests (625 complete-prior and 679 incomplete-prior), and 36 model/environment cells across 18 models,

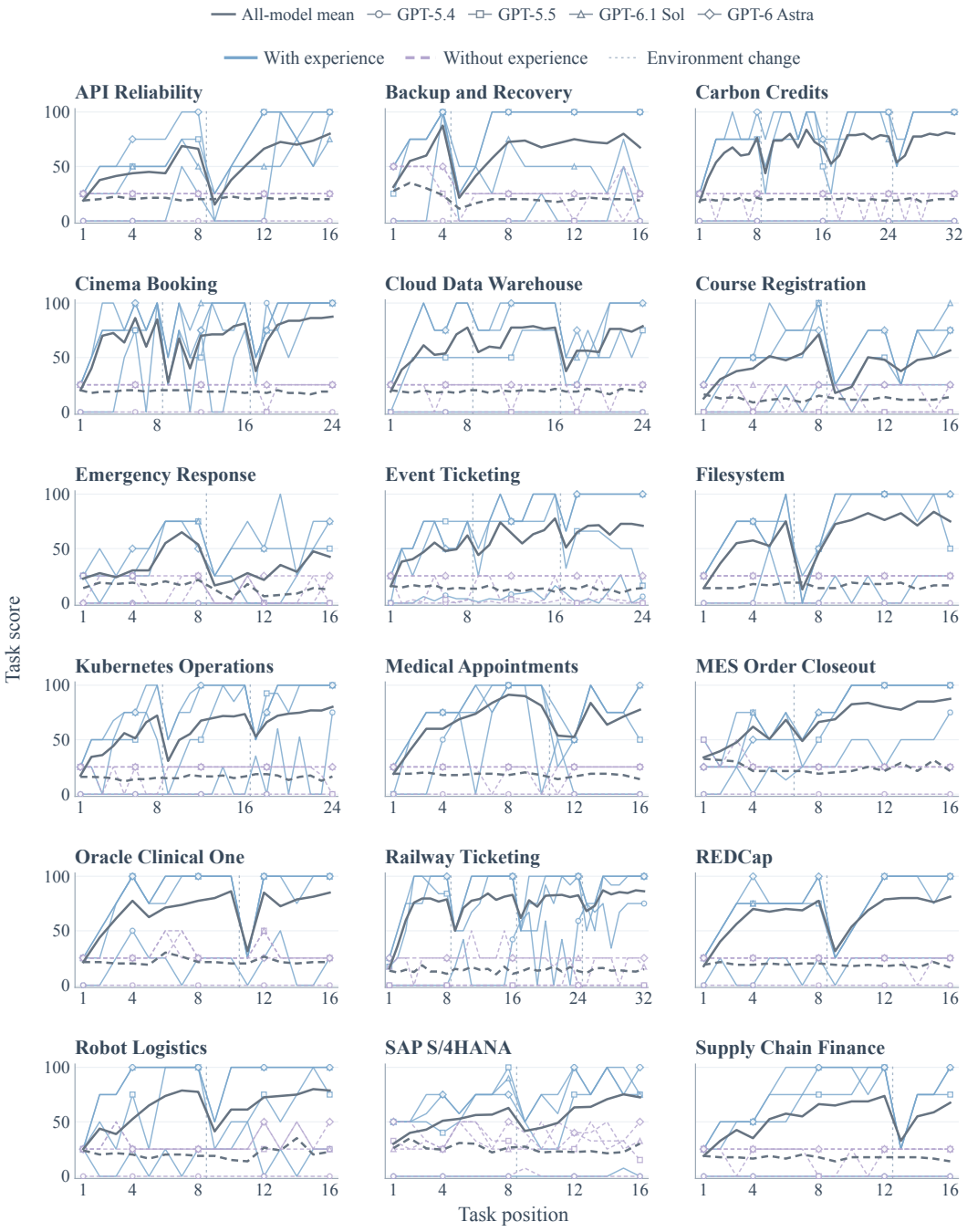


Figure 7: Task performance in all 18 environments. Solid and dashed curves show performance with and without experience. Vertical lines mark changes.

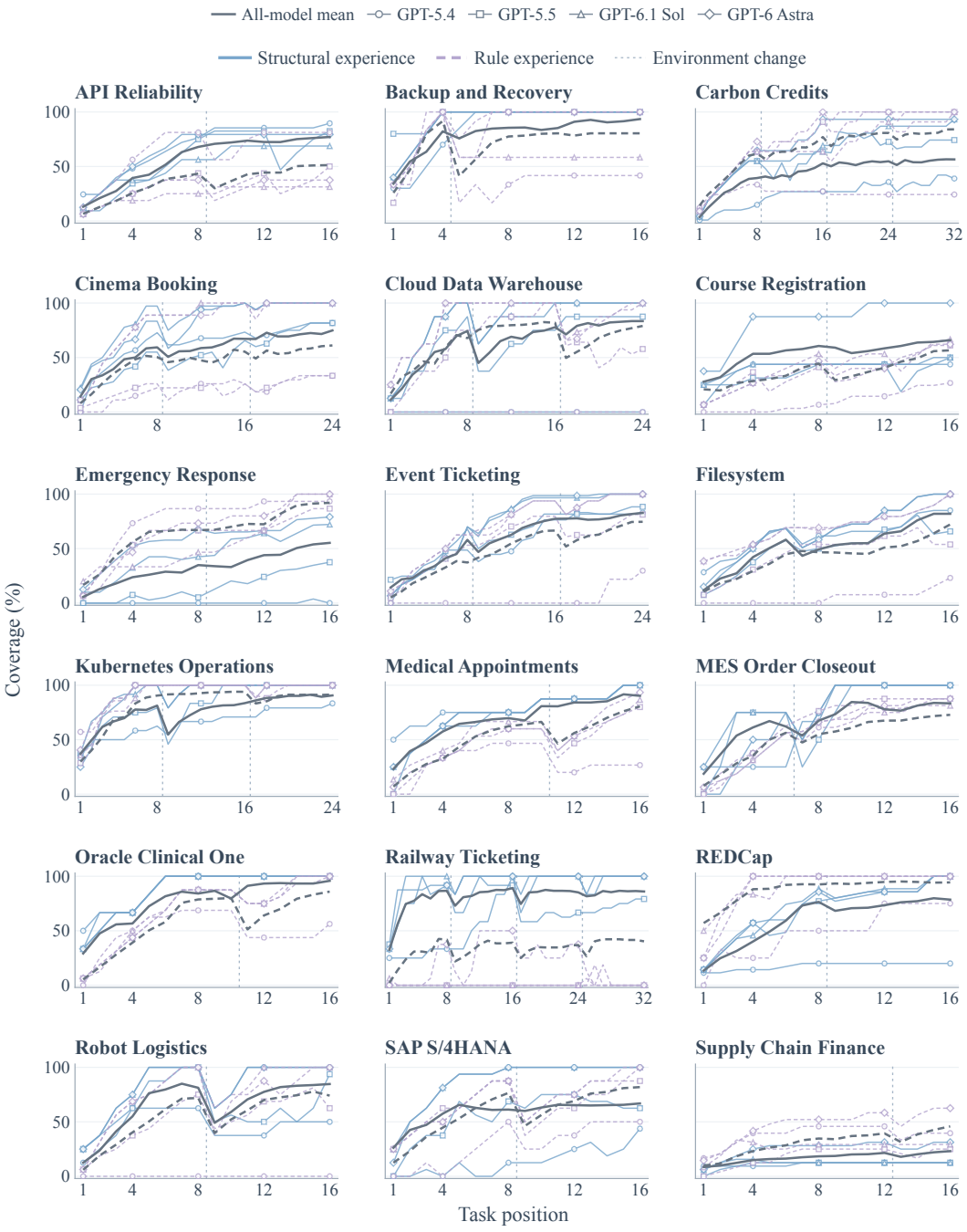


Figure 8: Post-task experience coverage in all 18 environments. Solid and dashed curves show structural and rule coverage. Vertical lines mark changes.

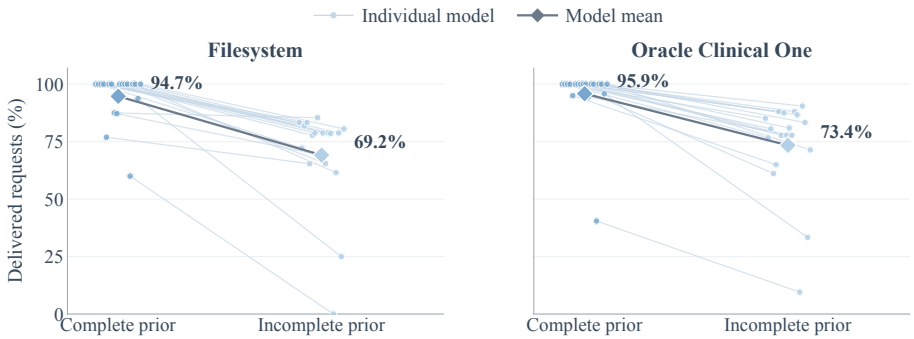


Figure 9: Delivery by prior-content completeness within matched tasks. Lines connect each model's two rates, and diamonds mark equal-model means.

Table 6: Task-matched request outcome rates (%) by completeness of prior experience.

Outcome	Complete prior	Incomplete prior
No linked submission	3.67	23.09
Submitted, never compatible	0.63	5.21
Compatible, not delivered	0.40	0.42
Delivered	95.31	71.28

without selecting on delivery. We compute each group's rate within a task, then average over eligible tasks, models within each environment, and the two environments equally. Both groups share task cells and outer weights. Eligible tasks vary across models: 5–14 in Filesystem and 5–10 in Oracle.

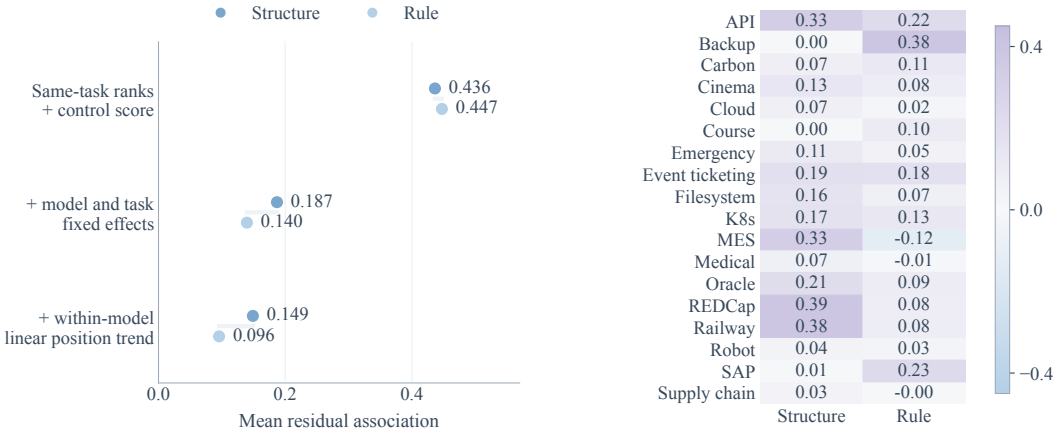
Figure 9 shows higher delivery with complete prior experience in all 36 model/environment cells. Complete- and incomplete-prior delivery rates average 94.74/69.16% in Filesystem and 95.88/73.40% in Oracle. The cross-environment rates are 95.31% and 71.28%, a 24.03-point gap. Requests without a linked submission account for 19.42 points, submissions that never become compatible for 4.58, and compatible submissions without delivery for 0.03. The gap therefore mainly concerns whether requests reach submission.

Table 6 separates four mutually exclusive outcomes. Compatibility requires all frozen structural and rule bindings, while delivery requires the resulting business artifact. An incompatible submission can be corrected before delivery. This decomposition is descriptive: matched requests can still differ in targets, ordering, remaining budget, and within-task discovery. In raw counts, 21 complete-prior requests fail delivery and 458 incomplete-prior requests succeed. Complete saved prior experience is thus neither necessary nor sufficient for delivery in this cohort.

C.1.3 Lagged Experience–Performance Associations

We next examine whether prior coverage remains associated with performance after accounting for model and task variation. We pair checkpoint $t-1$ with task t only within the same reference version. Excluding 360 initial and 520 change-onset observations leaves 6,160 observations at 308 environment/task positions. Unlike the request-specific prerequisites above, these predictors cover the full reference inventory.

Across model means, structural and rule coverage correlate strongly with with-experience performance (Pearson $r = 0.956/0.975$), but also with baseline performance ($0.898/0.928$). The coverage measures correlate at 0.965. We rank models within each task, average tied ranks, and adjust coverage and with-experience performance for contemporaneous baseline performance within each environment. We compute partial rank correlations per environment and average them equally. Figure 10 then adds model/environment and environment/task fixed effects, followed by model/environment-specific linear position trends, using the same observations.



(a) Adjustment sensitivity **(b) Environment variation**

Figure 10: Prior-coverage correlations with current performance over 6,160 same-version observations. (a) Equal-environment means under successive adjustments. (b) Fully adjusted per-environment coefficients.

The structural/rule associations fall from $0.436/0.447$ after baseline adjustment to $0.187/0.140$ with fixed effects and $0.149/0.096$ with position trends. The fully adjusted raw-score counterparts are $0.137/0.124$. Much of the aggregate association thus overlaps with differences among models, tasks, and stream progression. These descriptive adjustments do not identify a causal effect of saved experience. Residualizing either W or $W - B$ on B yields the same residual, so the two adjusted results provide the same evidence.

C.1.4 Acquisition and Loss of Previously Credited Content

To decompose the structural–rule coverage gap in Section 3.2, we separate claims never credited from those credited earlier but not currently. We use all 20 models in the 15 environments unchanged through T08, excluding Backup and Recovery, Filesystem, and MES Order Closeout. The reference version and cohort remain fixed throughout T01–T08, with the same 300 model/environment observations at every position.

For a reference claim c, let $z_{c,t} \in \{0, 1\}$ be its fixed post-task claim score and let $t_0(c)$ denote the start of its reference epoch. Its earlier-credit indicator is $a_{c,t} = \max_{t_0(c) \le s < t} z_{c,s}$, with an empty maximum equal to zero. The mutually exclusive states are

$$C_{c,t} = z_{c,t}, \qquad U_{c,t} = (1 - z_{c,t})(1 - a_{c,t}), \qquad L_{c,t} = (1 - z_{c,t})a_{c,t}. \tag{5}$$

Here, C denotes currently credited content, U content never credited in the current reference epoch, and L content credited earlier but not currently. Claim identities include the reference version, so reference changes do not count as state transitions. Lost credit alone does not establish deletion of saved content or model forgetting.

We average claim states within units, units within experience type, environments within model, and models equally. In percentage points, the coverage gap decomposes as

$$\mathrm{SC} - \mathrm{RC} = (U_R - U_S) + (L_R - L_S). \tag{6}$$

The first term compares never-credited content, and the second compares previously credited content that has lost credit. Table 7 shows that the early rule shortfall mainly reflects never-credited content. By T08, the gap narrows, but both types still contain substantial uncredited content: never-credited shares are 31.09% for structure and 32.86% for rules, and lost-credit shares are 5.33% and 5.74%. Structural and rule claims are not matched for semantic difficulty or evidence exposure.

Table 7: Structural-minus-rule coverage gap across 15 fixed environments and 20 models, in percentage points. Negative values favor rule coverage.

Checkpoint	$U_R - U_S$	$L_R - L_S$	SC − RC
T04	$+4.725$	-0.662	$+4.06$
T08	$+1.765$	$+0.417$	$+2.18$

Table 8: Sensitivity to the pre-change window, in score points. Each row uses a fixed cohort. Adjusted values subtract each event's pre-change mean.

Comparison	Raw Δ	Pre-1	Pre-2	Pre-3
All 26 events: first-four-task mean	37.74	-17.87	-14.58	-12.36
All 26 events: fourth task only	50.95	-4.66	-1.37	$+0.85$
T05 cohort: fourth task only	52.50	-11.25	$+5.63$	$+14.58$
Event 2 minus 1: eight-task mean, six environments	$+4.53$	$+2.38$	-1.14	-3.06
Event 3 minus 2: eight-task mean, two environments	$+2.78$	-1.39	-1.18	$+1.25$

C.2 Environmental Change and Experience Maintenance

C.2.1 Change Contexts and Baseline Sensitivity

To assess adaptation, we compare change schedules and mechanisms across 26 events, 18 environments, and all 20 models, then test sensitivity to the pre-change baseline. Here, W and B are task scores with and without experience, and $\Delta = W - B$. For an event at position d, offset k denotes task $d+k$. Offset $k = 0$ is the completed first affected task, including within-task adaptation. Mechanism comparisons use $k = -3, \ldots, 3$. The first-four-task mean averages $k = 0, 1, 2, 3$, unlike the fourth-task endpoint. Pre-n averages $k = -n, \ldots, -1$, for $n = 1, 2, 3$. Adjusted contrasts subtract each event's pre-n value before aggregation. Positive onset loss subtracts offset zero from pre-n.

For the five single-event schedule groups in Table 4, first-four-task Δ means at T05/T07/T09/T11/T13 are 31.25/40.94/27.05/45.78/37.19 points. Their complete post-event sequences contain 12/10/8/6/4 tasks. Figure 11 plots the groups separately to keep membership fixed across offsets. The event-order comparison uses the same six environments and 120 model/environment pairs for first and second events, with eight post-event tasks each. These observed schedules do not hold change targets and prior state fixed while varying onset.

We also group events by the mechanisms in Appendix A, which differ from structural and rule experience types. Event/environment counts are 11/9 for rule changes, 4/4 for structure, 3/3 for permission, 4/2 for resource, 2/2 for tool-path, and 2/2 for mixed changes. Environments can appear in multiple groups. Authors assign labels from change specifications with AI-assisted review. These labels describe observed contexts, not factorially controlled conditions.

Mechanism curves average events within each model/environment/mechanism, then environments within models and models equally. Environments thus receive equal weight regardless of event count. All seven offsets share the same cohort. Table 8 shows that the baseline-adjusted first-four-task contrast is negative under all three pre-change windows, while the adjusted fourth-task endpoint and event-order contrasts change sign. Figure 11 shows W, B, and Δ for the same observations. Figure 12 shows sensitivity to the baseline window and environment.

C.2.2 Selective Revision of Previously Correct Experience

To distinguish revision from first acquisition, we track relations credited immediately before a change. A model/unit/event case is eligible when every changed target has a credited one-to-one predecessor for the same semantic relation and at least one unchanged target is credited. The preserved set contains only these previously credited unchanged targets and remains fixed. This yields 487 cases across 20 models, eight environments, and 11 events.

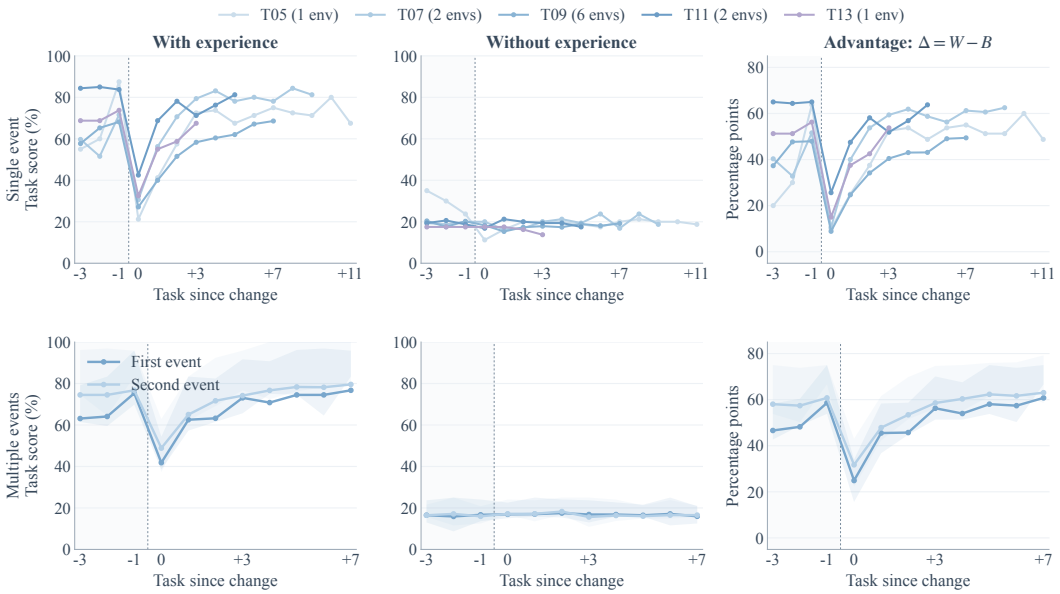


Figure 11: Performance and experience advantage across fixed change-schedule cohorts. Gray shading marks pre-change tasks. Lower-row bands show model interquartile ranges.

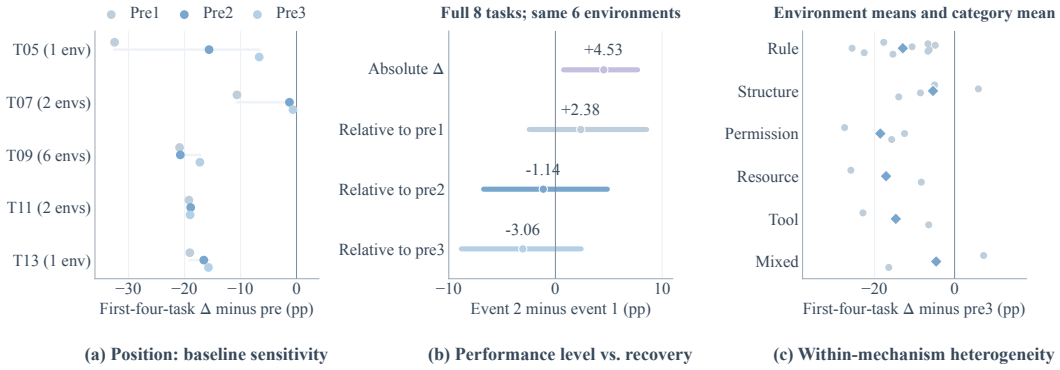


Figure 12: Sensitivity to the pre-change window and environment. (a) Timing-group means. (b) Model means and interquartile ranges. (c) Environment means (gray points) and mechanism-group means (diamonds).

For a changed or preserved target set A, coverage at post-event offset k is

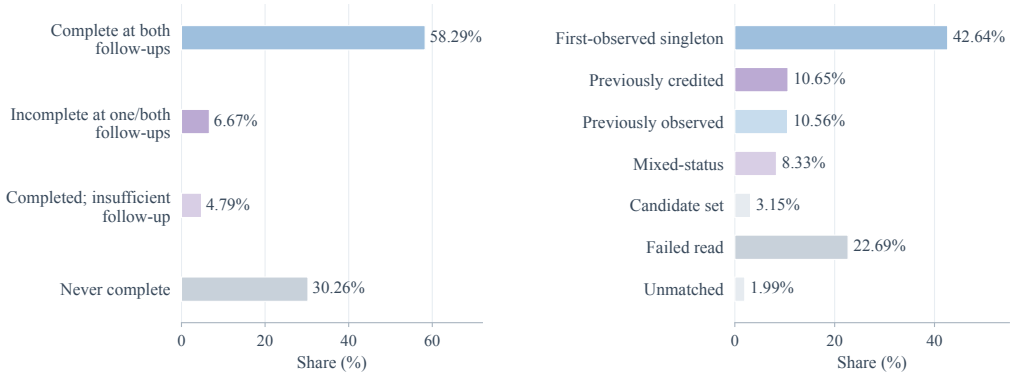
$$C_{A,k} = 100 \frac{\sum_{a \in A} w_a c_{a,k}}{\sum_{a \in A} w_a}, \tag{7}$$

where $c_{a,k}$ is current-target credit and w_a is the reviewed lineage weight, normalized within the target set. We average units within events, events within model/environment pairs, eligible environments within models, and models equally. Membership and weights remain fixed across the four offsets. Case eligibility can differ across models because it depends on prior acquisition. Many-to-many replacements and partitioned targets, including Railway pool replacements and Cloud partitioning, lack one-to-one predecessors and are excluded, not scored as failures.

Updated-content coverage rises from 21.47% to 43.07%, 59.40%, and 64.63% over the first four affected tasks. Preserved-content coverage is 89.05%, 88.07%, 90.38%, and 91.07%. At the fourth task, preservation exceeds updating in 19 of 20 model means and all eight environment means. Revision thus lags retention in this cohort. These trajectories follow the actual task schedule, and changed and preserved targets can differ in complexity and when they become relevant or observable.

Table 9: Replacement evidence and saved-content states in Filesystem. Each column partitions the same 122 model–relation pairs.

Revised-content state	T07	T08	T09	T10
Currently credited	13	59	91	92
Exposed, never credited	8	9	11	12
Exposed, previously credited but now uncredited	0	0	6	14
No mapped read	101	54	14	4



(a) Completion and follow-up **(b) Public-read categories**

Figure 13: Update persistence and read categories. (a) Completion and two-task follow-up for 2,340 affected model/unit/event observations. (b) Evidence from 2,160 business reads in Filesystem and Oracle. Bars show within-panel proportions.

To examine revision after replacement evidence becomes visible, we inspect the Filesystem subset: 122 changed model–relation pairs in 70 eligible model/unit/event cases across 20 models. We match each replacement binding to a successful `list_delivery_lanes` response by workspace, lane, and destination, then check its existing post-task claim score. Exposure accumulates within T07–T10 before each checkpoint. Table 9 partitions this fixed cohort using raw counts, unlike the macro-averaged coverage above. "No mapped read" does not exclude evidence from other channels, such as history or successful writes.

Of the 118 pairs exposed by T10, 104 receive credit at the checkpoint of their first matched read. Yet 26 exposed pairs lack credit at T10: 12 have never received it, and 14 received it earlier but lost it. Saved records illustrate two failures. At T07, Qwen3.5-35B-A3B records new Ember lane identifiers in memory while retaining conflicting old identifiers in a skill. Qwen3.8-Max records and receives credit for the new identifiers at T08, but omits them at T10. Observing replacements therefore does not ensure that saved experience stays current and internally consistent.

C.2.3 Persistence after First Completion

RT records first completion but not persistence. We therefore follow all 2,340 affected model/unit/event observations, including those outside the prior-correct cohort. Completion requires simultaneous credit for all current update targets N and the fixed unchanged targets V credited before the event. With empty V, it measures update completion only. We track each first completion over two further checkpoints within the same reference epoch.

Of 1,632 observations that complete at least once, 1,520 have both follow-ups. Among them, 1,364 remain complete at both checkpoints and 156 (10.26%) become incomplete at one or both. Another 112 completions lack sufficient follow-up, and 708 observations never complete. Figure 13(a) shows all four outcomes. The 10.26% regression rate uses only the 1,520 eligible completions. First completion thus does not guarantee completeness over the next two tasks. Regression here means lost credit, not necessarily permanent forgetting or impaired task performance.

C.3 Interaction and Action Allocation

To examine the actions behind request outcomes, we use the same Filesystem and Oracle trace cohort. Its 8,826 foreground calls comprise 2,189 business executions, 2,160 business reads, 1,813 experience operations, and 2,664 history queries. Counts include failures but exclude automatic memory injection and background review.

First-submission paths use only evidence visible before the assistant turn that generates the arguments. A parallel read in that turn cannot inform them. We classify requests by first-submission compatibility and whether a successful response for their scope preceded submission, retaining no-submission and unknown categories. The matched cohort has no unknown cases. Figure 5(a) merges the two incompatible-submission paths. A response within a request's scope need not resolve every required binding.

Read categories describe returned evidence, not intent. A *first-observed singleton* supplies one accepted current binding with no earlier singleton or candidate-set exposure through the instrumented interfaces and no same-version prior credit. A *prior-credited query* returns a binding credited at checkpoint $t - 1$. A *re-observed singleton* repeats an earlier visible binding when prior credit is absent or unavailable. The latter two categories are *re-queries*. Candidate sets include the accepted value among alternatives, while mixed responses contain multiple statuses. Failed and unmatched reads remain separate. Earlier evidence carries across a change only if its type, unit, condition, field, and complete accepted-value set are unchanged.

Figure 13(b) counts each business read once: 42.6% are first-observed singletons and 21.2% are re-queries.

The main-text trajectories average calls per task across all 20 models, including tasks with no reads. Across complete pre-change and post-change epochs, first-observed singletons fall from 2.53 to 1.14 per task in Filesystem and from 1.86 to 0.15 in Oracle. The epochs are T01–T06 versus T07–T16 in Filesystem and T01–T10 versus T11–T16 in Oracle. Re-queries rise from 0.39 to 1.08 and from 0.47 to 0.84, respectively. These means weight tasks equally, unlike pooled call proportions, and do not isolate the effect of change.

In the matched 1,304-request cohort across 18 models, 201 requests receive no linked submission: 133 in Filesystem and 68 in Oracle. We inspect terminal resource receipts and states to check whether omissions coincide with exhausted budgets. Filesystem capacity is the smaller of remaining write attempts and business-call slots. Oracle capacity is the shared balance divided by the two-unit correction cost, rounded down. All 133 Filesystem omissions retain capacity for a write attempt per omitted request. In Oracle, 61 of 68 omissions leave insufficient budget for even one correction. This contrast also holds with complete prior bindings: all 10 Filesystem omissions retain write capacity, while all six Oracle omissions do not. The eight Filesystem tasks containing those 10 omissions end normally below the model-turn limit.

These balances describe terminal resources, not availability at decision time or delivery under alternative actions. Appendix D.2 illustrates the trade-off between checking and execution at Oracle T10. Call counts exclude tokens, latency, and background work. Domain-specific resource units are not pooled.

A later Oracle trace shows that successful delivery can coexist with an incorrect explanation of change. At T14, DeepSeek-V4-Pro-0813 retains a skill stating that policy validity "flips per episode," although the four affected cell policies change once at T11 and remain fixed thereafter. Trying two candidate configurations completes all four corrections in six write attempts, including two policy mismatches. This uses 12 of 17 units and scores 100 without new business reads. The fallback succeeds despite the saved skill's incorrect explanation.

D ILLUSTRATIVE CASE STUDIES

D.1 LOCAL REVISION AND SUBSEQUENT REUSE

Figure 14 follows GPT-5.5's Riverlight Ledger binding after a silent destination change before T07. At T09, the agent rechecks public evidence and updates the directory handle while preserving five valid executable fields. The revised binding supports five later deliveries (T11–T14 and T16) without further business reads for the manifest.

The case also contrasts the two scoring procedures. The experience claim specifies the directory for Grove's amber lane. Applying the reference rubric to saved content, the LLM judge scores it 0 at T07 and 1 at T09. The programmatic task grader instead checks the delivered artifact and final state. Its five checks award the selected T09 request 25/25 points, while the full task scores 75/100.

Task and environment: Filesystem

Task instruction. Materialize as many of the four requested manifest indexes as possible in any order. Resolve each current manifest and its workspace delivery-lane handles, obey the current delivery policy, and place the generated index in the exact eligible lane. A successful materialize_manifest receipt confirms completion; no extra readback is required. Retain exact manifest identity, workspace-to-lane, and policy bindings so a recurring manifest can be executed without redundant reads unless ordinary tool evidence shows that a binding is stale.
Selected request. request-ac8e9d302a3ddf407c1f: workspace=Grove Evidence Workspace; manifest=Riverlight Ledger.

Business tools: `list_manifest_requests, list_delivery_lanes, get_delivery_policy, materialize_manifest`. **Experience:** `memory`. **History:** `record_search, record_read` (available; not used in this repair).

Saved experience after T04 (GPT-5.5, MEMORY.md)

Manifest binding verified 2026-10-13: Grove Evidence Workspace / Riverlight Ledger (request-ac8e9d302a3ddf407c1f) -> workspace_id workspace-g9c-k4, manifest_id manifest-ce51320525aac548fc5ad77d, delivery_lane amber, **lane_node_id node-lane-k2d8s4**, routing_token route-74c8905c6648f84c622b3bc5, overwrite_existing false. Materialized node-g9c-k4-riverlight-ledger-batch-fa25c71c8a4e-index-md (op-g9c-k4-2026-05-create-0009).

Observed failure and saved warning at T07

Attempt. `materialize_manifest`

```
{"workspace_id": "workspace-g9c-k4", "manifest_id": "manifest-ce51320525aac548fc5ad77d",
"destination_parent_node_id": "node-lane-k2d8s4", "routing_token":
"route-74c8905c6648f84c622b3bc5", "overwrite_existing": false}
```

Tool response.

```
{"budget": {"read_work_units_remaining": 1, "read_work_units_used": 8,
"write_attempts_remaining": 6, "write_attempts_used": 3}, "data": {"error_code":
"DELIVERY_POLICY_MISMATCH"}, "message": "request does not satisfy the current manifest delivery
policy", "status": "error"}
```

Saved warning. Manifest stale binding observed 2026-10-13: Grove Evidence Workspace / Riverlight Ledger request-d0c5240cc43cadb2a67b with prior binding workspace-g9c-k4, manifest-ce51320525aac548fc5ad77d, amber/node-lane-k2d8s4, route-74c8905c6648f84c622b3bc5, overwrite_existing false returned DELIVERY_POLICY_MISMATCH; refresh current policy/lane before executing.

New evidence and revised experience at T09

New evidence. `list_delivery_lanes`: new amber handle; `get_delivery_policy`: unchanged policy.

Manifest binding verified 2026-10-13: Grove Evidence Workspace / Riverlight Ledger (requests include request-5308833fb86d5288570b; prior stale request-d0c5240cc43cadb2a67b) -> workspace_id workspace-g9c-k4, manifest_id manifest-ce51320525aac548fc5ad77d, delivery_lane amber, **lane_node_id node-lane-7b32ae244f92430dad1a7683**, routing_token route-74c8905c6648f84c622b3bc5, overwrite_existing false. Materialized node-g9c-k4-riverlight-ledger-batch-f36002234af6-index-md (op-g9c-k4-2026-05-create-0021).

Experience rubric: saved amber binding

Reference: `node-lane-7b32ae244f92430dad1a7683`

Rubric requirement	T07	T09
Grove workspace and amber lane	Pass	Pass
Lane-to-directory relationship	Pass	Pass
Current directory handle	Fail	Pass
Indirect support, if needed	N/A	N/A
Claim credit	**0**	**1**

Credit requires every applicable check to pass.

Task grader: Riverlight delivery at T09

Five checks on persisted records (summarized).

Required result	Points
One index linked to the materialized manifest	2.5
Current amber directory, path, and policy	5
Both source paths, exactly two entries	5
Exact text/hash, 265 bytes, six lines	5
Usage reconciled within quota, one creation	7.5
Request: 25/25	Whole task: 75/100

Subsequent reuse

At T11-T14 and T16, the revised binding supports delivery without a new business read for this request. All five artifact checks pass at each position.

Selected memory entries and the failed call are shown in full. Blue marks structural fields, purple marks rule fields, and bold marks the changed directory handle. The rubric panel shows one claim. GPT-6 Astra judges the complete experience unit against the full saved corpus.

Figure 14: Local revision and subsequent reuse in Filesystem. Selected complete records link the revised binding to experience judgments and delivery checks.

D.2 Using Saved Experience under a Budget

Figure 15 illustrates the execution difficulty in Section 3.3.3: complete saved experience may still yield incomplete delivery under a resource budget. At the start of Oracle Clinical One T10, all four models have recorded the required structural and rule bindings. Four corrections cost eight units, leaving room for one five-unit context read within the 17-unit budget. A second read makes full delivery unaffordable. GPT-5.4 spends 15 units on three reads and completes one correction. GPT-5.5 and Claude Opus 5.5 complete all four with eight units. Doubao-Seed-2.1-Pro does so after one read, using 13. GPT-5.4 and Doubao-Seed-2.1-Pro retain an inactivity warning about one requested record, providing context for checking. One targeted check remains compatible with full delivery, while further checks consume the budget needed for execution.

Task and resource budget: Oracle Clinical, T10 (before drift)

Task. "Apply the four requested completed-form corrections in any order. An unfinished correction does not block an unrelated form. Use the exact corrected value supplied for each request and leave a policy-compliant audit record."

Budget. 17 backend work units; context read: 5; correction write: 2. The four required structural and rule bindings are recorded in each model's prior experience.

Selected complete records available before T10

GPT-5.4: MEMORY.md

Structural binding.
Correction handle ch-353c64e648495b39d95a maps to record_ref cr-c64a097e in this environment instance; visit_type dispensation; handle reusable across episodes.

Rule binding.
Audit profile ap-3f6c successfully committed a coding_alignment correction on visit_type dispensation in this environment instance.

GPT-5.5: SKILL.md

- Resolve a `record_ref` with `get_correction_context` before writing **unless a retained binding** gives the exact `correction_handle`, visit type, and completed/saved status for the same environment instance. Context dossier joins cost 5 backend work units.

- Under tight budget, prioritize independent corrections that already have both a reusable handle and a verified policy profile; failed audit-profile guesses consume write budget.

Prior warning and current task context

GPT-5.4 saved warning. Observed write with remembered handle ch-a33e5e5e1d898823b9c4 for cr-18b5ea74 was rejected with CORRECTION_HANDLE_NOT_ACTIVE; do not assume retained handles stay active without current-task confirmation.

The record was not requested at T09 and is requested again at T10 with the same handle. Doubao retained a warning about the same record.

Observed actions and outcomes

Model	Charged business operations (in order)	Cost	Delivered
GPT-5.4	Read cr-18b5ea74, cr-a21d6e40, cr-c64a097e; write cr-c64a097e	17/17	1/4
GPT-5.5	Write cr-18b5ea74, cr-a21d6e40, cr-c64a097e, cr-7f3b912c	8/17	4/4
Claude Opus 5.5	Write cr-18b5ea74, cr-a21d6e40, cr-c64a097e, cr-7f3b912c	8/17	4/4
Doubao Seed 2.1 Pro	Read cr-18b5ea74; write cr-a21d6e40, cr-c64a097e, cr-7f3b912c, cr-18b5ea74	13/17	4/4

Selected memory entries and workflow bullets are reproduced in full; emphasis is added. Experience and history access have separate limits and are excluded from the business costs above.

Figure 15: Complete saved experience, resource use, and delivery on the same Oracle Clinical One task. Selected entries are reproduced in full.